

Uniwersytet Gdański
Wydział Matematyki, Fizyki i Informatyki

Dawid Kacper Odolczyk
nr albumu: 298988
kierunek studiów: informatyka

Analiza porównawcza i integracyjna
mechanizmów Project Loom
z istniejącymi modelami
współbieżności w środowisku JVM
w kontekście wybranych scenariuszy
zastosowań

Praca magisterska
wykonana pod kierunkiem
dra Tomasza Borzyszkowskiego

Gdańsk 2026

Spis treści

Streszczenie	7
1 Wstęp	9
1.1 Motywacja	9
1.2 Cel i zakres pracy	9
1.3 Pytania badawcze	9
1.4 Struktura pracy	9
2 Wprowadzenie teoretyczne	11
2.1 Podstawowe pojęcia	11
2.1.1 Współbieżność a równoległość	11
2.1.2 Zasada działania procesów i wątków	12
2.2 Problemy występujące w systemach współbieżnych	13
2.2.1 Race condition i pamięć współdzielona	13
2.2.2 Problemy żywotności: zakleszczenie, zagłodzenie, livelock	15
2.2.3 Problemy związane z wydajnością	17
2.3 Modele współbieżności w środowisku JVM	17
2.3.1 Klasyczny model współbieżności	18
2.3.2 Model współbieżności oparty o pule wątków	18
2.3.3 Reaktywny model współbieżności	20
3 Mechanizmy współbieżności oferowane przez Project Loom	23
3.1 Wprowadzenie do Project Loom	23
3.2 Wirtualne wątki (<i>Virtual Threads</i>)	24
3.2.1 Zasada działania i cykl życia wirtualnych wątków	25
3.2.2 Pule wirtualnych wątków	27
3.3 Współbieżność strukturalna (<i>Structured Concurrency</i>)	27
3.3.1 Cechy współbieżności strukturalnej	28
3.3.2 Struktura i działanie zakresów zadań	28
3.3.3 Strategie synchronizacji podzadań	30
3.4 Wartości zakresowe (<i>Scoped Values</i>)	30

3.4.1	Opis mechanizmu <code>ThreadLocal</code>	31
3.4.2	Zasada działania wartości zakresowych	31
4	Przegląd literatury i opis stanu badań nad Project Loom	33
4.1	Opis i charakterystyka badań nad Project Loom	33
4.2	Przegląd wybranych pozycji	34
4.2.1	Prace akademickie	34
4.2.2	Dokumentacja techniczna	35
4.3	Zdefiniowanie luki badawczej i kierunku własnych badań	35
5	Metodyka badań	37
5.1	Cel i zakres badań	37
5.2	Szczegółowy opis scenariuszy testowych	38
5.2.1	Scenariusz $\mathcal{A}$: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)	38
5.2.2	Scenariusz $\mathcal{B}$: Agregacja wyników oparta na mechanizmie kworum	44
5.3	Hipotezy badawcze	51
6	Wyniki badań i ich analiza	53
6.1	Kryteria oceny wyników	53
6.1.1	Opis użytych trybów benchmarków i definicja mierzonych przez nie metryk	53
6.1.2	Definicja metryk statycznej analizy kodu	54
6.2	Szczegółowy opis pomiarów	54
6.2.1	Szczegóły pomiaru dla scenariusza $\mathcal{A}$	54
6.2.2	Szczegóły pomiaru dla scenariusza $\mathcal{B}$	55
6.3	Prezentacja wyników pomiarów	55
6.3.1	Wyniki pomiarów w ramach scenariusza $\mathcal{A}$	55
6.3.2	Wyniki pomiarów w ramach scenariusza $\mathcal{B}$	68
6.4	Analiza porównawcza i interpretacja wyników	72
6.4.1	Scenariusz $\mathcal{A}$	72
6.4.2	Scenariusz $\mathcal{B}$	72
7	Wnioski i rekomendacje	73
7.1	Odpowiedzi na pytania badawcze i weryfikacja hipotez badawczych	73
7.2	Rekomendacje i wzorce zastosowań mechanizmów Project Loom	73
7.3	Ograniczenia badań i propozycje dalszych badań	73
8	Podsumowanie	75

Bibliografia	77
Spis rysunków	82
Spis tabel	84
Spis kodów źródłowych	85

Streszczenie

STRESZCZENIE

Rozdział 1

Wstęp

Pierwszy rozdział pracy wskazuje motywację niniejszej pracy, jej cel i zakres, pokrótce opisuje pytania badawcze, będące podstawą do zdefiniowania hipotez badawczych i określa strukturę pracy.

1.1 Motywacja

1.2 Cel i zakres pracy

1.3 Pytania badawcze

1.4 Struktura pracy

Rozdział 2

Wprowadzenie teoretyczne

Drugi rozdział pracy poświęcony jest teoretycznemu omówieniu podstawowych pojęć związanych z równoległością i współbieżnością, w tym w omówieniu działania wątków we współczesnych systemach operacyjnych, zdefiniowaniu typowych problemów występujących w systemach współbieżnych oraz przedstawieniu najważniejszych założeń różnych modeli współbieżności dostępnych w ramach środowiska JVM, w szczególności klasycznego modelu współbieżności oferowanego w ramach Thread API, modelu opartego o pulę wątków (dostępnego w ramach interfejsu `ExecutorService`) oraz reaktywnego modelu współbieżności.

2.1 Podstawowe pojęcia

2.1.1 Współbieżność a równoległość

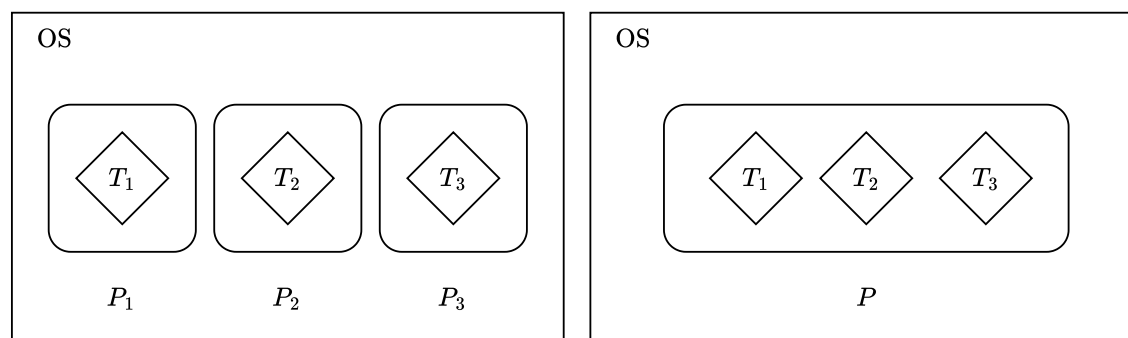
Jak podają Tanenbaum i Bos [1], *współbieżność* (ang. *concurrency*) oznacza zdolność systemu do obsługi wielu zadań w taki sposób, że ich wykonywanie nakłada się w czasie, nawet jeśli w danym momencie wykonywane jest tylko jedno z nich. *Równoległość* (ang. *parallelism*) oznacza obsługę wielu zadań w tym samym czasie przy wykorzystaniu wielu jednostek obliczeniowych.

Innymi słowy, współbieżność odnosi się do strukturalnej organizacji systemu oraz sposobu modelowania i zarządzania wieloma zadaniami, natomiast równoległość oznacza ich faktyczne, jednoczesne wykonywanie na wielu procesorach lub rdzeniach. We współczesnych systemach operacyjnych zadania te są wykonywane w ramach *procesów i wątków*.

2.1.2 Zasada działania procesów i wątków

We współczesnych systemach operacyjnych najważniejszą abstrakcją umożliwiającą współbieżne wykonywanie operacji są *procesy* (ang. *processes*). Jak wskazują Tanenbaum i Bos [1], procesy umożliwiają przekształcenie jednej jednostki obliczeniowej (CPU) na wiele wirtualnych jednostek, w ramach których mogą współbieżnie wykonywać zadania poprzez umiejętność CPU do szybkiego przełączania się między procesami.

W celu umożliwienia bardziej efektywnej współbieżności wewnątrz pojedynczego procesu, współczesne systemy operacyjne oferują *wątki* (ang. *threads*), definiowane jako lżejsze jednostki wykonania. Wątki działają w obrębie jednego procesu i współdzielą z nim przestrzeń adresową oraz zasoby, zachowując jednocześnie własny kontekst wykonania [1]. Różnice między procesem a wątkiem prezentuje rysunek 2.1.



(a) trzy jednowątkowe procesy

(b) jeden trzywątkowy proces

Rysunek 2.1: Rozróżnienie procesów i wątków w systemie operacyjnym

(źródło: opracowanie własne na podstawie [1])

Mimo że oba systemy przedstawione powyżej operują na trzech wątkach, wątki na rysunku 2.1a korzystają z odrębnych przestrzeni adresowych, podczas gdy wątki przedstawione na rysunku 2.1b działają w obrębie jednej (wspólnej) przestrzeni adresowej, co wynika z faktu operowania wewnątrz jednego (wspólnego) procesu P .

Wprowadzenie wątków jako lekkich jednostek wykonania znacząco zwiększa efektywność wykorzystania zasobów systemowych, jednak jednocześnie wprowadza nowe wyzwania związane z koniecznością synchronizacji dostępu do współdzielonych danych. Współdzielenie przestrzeni adresowej przez wątki powoduje ryzyko występowania różnego rodzaju zjawisk, opisanych szczegółowo w podrozdziale 2.2 Problemy występujące w systemach współbieżnych. Z tego względu poprawne projektowanie systemów wielowątkowych wymaga nie tylko zrozumienia modelu wykonania wątków, ale również potencjalnych problemów wynikających z ich współbieżnej natury.

2.2 Problemy występujące w systemach współbieżnych

Mimo wielu zalet wykorzystania wątków do współbieżnego wykonywania zadań, podejście to wiąże się z istnieniem wielu problemów obecnych w systemach współbieżnych. Ben-Ari [2] wskazuje zasadę *wzajemnego wykluczenia* (ang. *mutual exclusion*) jako jeden z najważniejszych elementów programowania współbieżnego z racji bycia abstrakcją dla wielu problemów związanych z zarządzaniem wątkami. Zgodnie z definicją Herlihy’ego i Shavita [3], wątki *wykluczają się wzajemnie*, jeśli co najmniej dwa z nich próbują równocześnie wykonać wspólny blok kodu, nazywany *sekcją krytyczną* (ang. *critical section*), ale w danym momencie tylko jeden z nich ma do niej dostęp. Tę cechę systemu określa się również mianem *synchronizacji* wątków [4]. Oznacza to, że dany wątek ma dostęp do sekcji krytycznej tylko wtedy, gdy inny wątek zakończy w niej swoje operacje.

Zapewnienie tej cechy jest istotne w kontekście poprawności i jednoznaczności wyniku wielowątkowego zadania, a jej brak powoduje występowanie wielu kategorii problemów. Wśród nich można wyróżnić: problem *race condition* i pamięci współdzielonej, problemy żywotności, takie jak *zakleszczenie* (ang. *deadlock*), *zagłodzenie* (ang. *starvation*) i *livelock*, a także problemy związane z wydajnością. Poniżej opisano charakterystykę każdego z nich.

2.2.1 Race condition i pamięć współdzielona

Goetz [5] definiuje problem *race condition* (w polskiej literaturze często określany mianem *warunku wyścigu* lub *stanu wyścigu* [6, 7]) jako zachowanie systemu współbieżnego, w którym poprawność obliczeń zależy od względnego czasu lub przeplatania wielu wątków przez środowisko wykonawcze. Innymi słowy, problem ten występuje, gdy uzyskanie oczekiwanego rezultatu jest zależne od zbiegu okoliczności. Najczęstszym rodzajem tego problemu jest sytuacja, w której potencjalnie nieaktualna wartość jest wykorzystywana do dalszych obliczeń, co może prowadzić do niezamierzonych rezultatów. Poniższy przykład szczegółowo prezentuje to zjawisko.

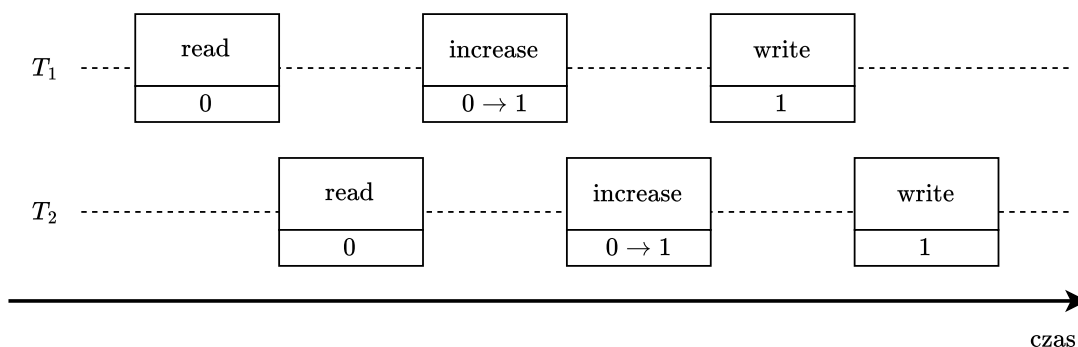
Niech procedura `increaseByOne` składa się z trzech wykonywanych sekwencyjnie operacji:

1. odczytania wartości zmiennej globalnej (ozn. `read`),
2. zwiększenia odczytanej wartości o 1 (ozn. `increase`),
3. zapisania nowej wartości do zmiennej globalnej (ozn. `write`).

Procedurę tę uruchomiono współbieżnie z wykorzystaniem wątków T_1 i T_2 dla tej samej wartości początkowej, tj. pewnej zmiennej globalnej G o wartości początkowej

równej 0. Przewidywanym wynikiem dwukrotnego wywołania procedury na zmiennej G jest zwiększenie wartości tej zmiennej do 2.

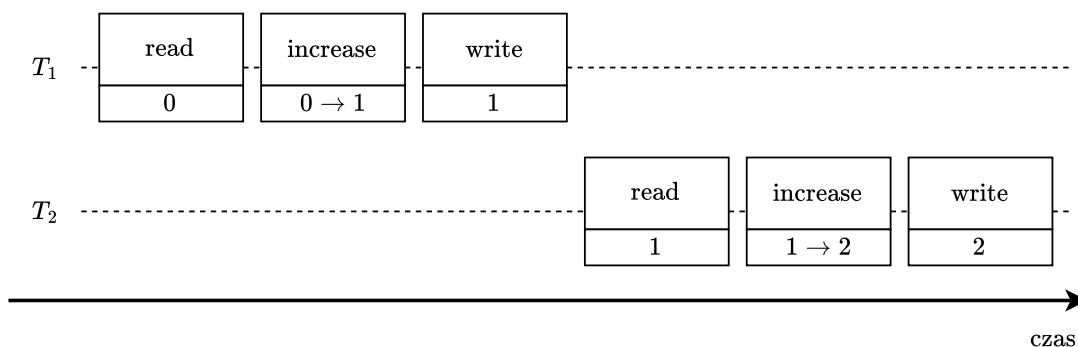
Rysunki 2.2 i 2.3 przedstawiają różne modele wywołań procedury `increaseByOne` na wątkach T_1 i T_2 . Prezentują one wywołania poszczególnych operacji tej procedury dla poszczególnych wątków w zależności od czasu. Każda operacja zawiera dwie informacje: nazwę oraz aktualną wartość zmiennej globalnej G . Wyjątek stanowi operacja `increase`, dla wartości której przyjęto oznaczenie „ $A \rightarrow B$ ”, gdzie A oznacza wartość zmiennej G przed dokonaniem na niej operacji zwiększenia wartości o 1 (tj. jej aktualna wartość), a B oznacza wartość zmiennej G po wykonaniu na niej operacji.



Rysunek 2.2: Ilustracja zjawiska *race condition* na podstawie wywołania procedury `increaseByOne`

(źródło: opracowanie własne na podstawie [5])

Rysunek 2.2 prezentuje jeden z przypadków wystąpienia zjawiska *race condition*, w którym poszczególne operacje procedury `increaseByOne` przeplatają się w czasie. W rezultacie wątek T_2 odczytał wartość zmiennej globalnej G w momencie, gdy nie została ona jeszcze zaktualizowana przez wątek T_1 , co w konsekwencji spowodowało, że wartość zmiennej G wynosi 1 zamiast przewidywanej wartości 2. Rozwiązaniem tego problemu jest wcześniej wspomniana synchronizacja wątków T_1 i T_2 .



Rysunek 2.3: Wywołanie procedury `increaseByOne` z wykorzystaniem synchronizacji wątków

(źródło: opracowanie własne na podstawie [5])

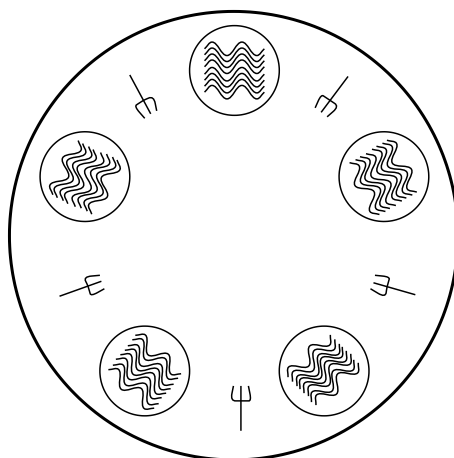
Rysunek 2.3 przedstawia wywołanie procedury `increaseByOne` na obu zsynchronizowanych wątkach, tj. wątek T_2 wykonał przydzielone mu operacje dokładnie wtedy, gdy wątek T_1 wykonał wszystkie swoje operacje. W rezultacie wartość zmiennej globalnej G wynosi 2, ponieważ procedura wywołana w wątku T_1 zwiększyła wartość zmiennej G o 1, a następnie to samo wykonała procedura wywołana w wątku T_2 , wykorzystując wynik operacji wykonanych przez wątek T_1 .

2.2.2 Problemy żywotności: zakleszczenie, zagłodzenie, livelock

Goetz [5] definiuje *żywotność* (ang. *liveness*) jako właściwość systemu, która zapewnia, że koniec końców pewna własność się wydarzy, czyli że system nie będzie pozostawał w stanie zawieszenia lub bezczynności. Problemy żywotności występują, gdy system nie jest w stanie kontynuować wykonywania zadań z powodu wzajemnych zależności między wątkami lub braku zasobów. Najpopularniejszymi z nich są *zakleszczenie*, *zagłodzenie* i *livelock*.

Problem uczających filozofów

Klasyczną ilustracją problemów żywotności, często przytaczaną w literaturze, jest wymyślony w 1965 roku przez Edsgera Dijkstrę *problem uczających filozofów* (ang. *dining philosophers problem*) [8]. Opisuje on pięciu filozofów siedzących przy okrągłym stole. Każdy filozof ma przed sobą talerz z posiłkiem, a między każdą sąsiadującą parą talerzy znajduje się widelec. Filozof może wykonywać jedną z dwóch czynności: albo je, albo rozmyśla. Zakłada się, że filozof może jeść tylko przy użyciu dwóch widelców. Dodatkowo, filozof może korzystać jedynie z widelców sąsiadujących z jego talerzem [1, 2].



Rysunek 2.4: Ilustracja problemu uczających filozofów

(źródło: opracowanie własne na podstawie [1])

W powyższym problemie poszczególne jego elementy stanowią abstrakcyjne reprezentacje składników systemu współbieżnego, tj. filozofowie odpowiadają procesom lub wątkom wykonującym się równolegle, które cyklicznie przechodzą pomiędzy sekcją niekrytyczną (rozmyślanie) a sekcją krytyczną (jedzenie), zaś widelce reprezentują współdzielone zasoby, do których dostęp musi być synchronizowany.

Zakleszczenie

Zakleszczeniem (ang. *deadlock*) określa się sytuację w systemie współbieżnym, w której co najmniej dwa procesy lub wątki czekają na siebie nawzajem w taki sposób, że żaden z nich nie może kontynuować wykonania [5].

Przykładem zakleszczenia w powyższym problemie jest sytuacja, w której każdy filozof bierze do jednej ręki widelec po lewej stronie swojego talerza. Aby móc zacząć jeść, filozof potrzebuje drugiego widelca, lecz każdy z nich nie posiada wolnego widelca po prawej stronie swojego talerza. Powoduje to wzajemną zależność czekania na zwolnienie zasobu, a w konsekwencji żaden z filozofów nie przystąpi do jedzenia.

Zagłodzenie

Zagłodzeniem (ang. *starvation*) wątku nazywa się sytuację, w której wątek jest stale pozbawiony dostępu do wspólnego zasobu, przez co nie może wykonać przydzielonych mu operacji [5].

Przykładem zagłodzenia w kontekście problemu uczujących filozofów jest sytuacja, w której jeden filozof nigdy nie może przystąpić do jedzenia, ponieważ sąsiadujący z nim filozofowie zawsze wygrywają w walce o wspólny zasób, jakim jest widelec (zarówno ten po lewej, jak i po prawej stronie talerza).

Livelock

Livelock jest stanem systemu współbieżnego, w którym procesy nie są zablokowane i wykonują operacje, jednak wskutek ciągłych wzajemnych reakcji nie dochodzi do postępu w realizacji ich zadań [5].

Livelock w kontekście wspomnianego wyżej problemu występuje, gdy filozofowie, próbując uniknąć zakleszczenia, zwalniają posiadane zasoby w przypadku niepowodzenia i ponawiają próby w sposób symetryczny, co prowadzi do cyklicznego wzorca zachowań. W rezultacie system pozostaje aktywny, jednak żaden z procesów nie uzyskuje jednocześnie wymaganych zasobów, przez co nie dochodzi do postępu.

2.2.3 Problemy związane z wydajnością

Goetz [5] wskazuje, że równie ważną właściwością systemu współbieżnego jak żywotność jest *wydajność* (ang. *performance*). Podczas gdy żywotność zapewnia, że koniec końców pewna własność się wydarzy, pojęcie wydajności obejmuje szerszy zakres – uwzględnia ono wykonywanie jak największej liczby zadań przy wykorzystaniu zasobów jak najmniejszym poziomem. Problemy związane z wydajnością obejmują szeroki zakres problemów, w tym krótki czas obsługi, responsywność, przepustowość, zużycie zasobów czy skalowalność. Podobnie jak w przypadku bezpieczeństwa i żywotności, programy wielowątkowe są narażone na wszystkie zagrożenia wydajnościowe programów jednowątkowych, a także na inne, które wynikają z użycia wątków. Jednym z najistotniejszych problemów związanych z wydajnością jest problem nadmiernego przełączania kontekstu.

Przełączanie kontekstu

Mianem *przełączania kontekstu* (ang. *context switching*) określa się mechanizm systemowy polegający na wstrzymaniu wykonywania jednego wątku oraz przywróceniu stanu innego wątku, umożliwiając współdzielenie zasobów procesora pomiędzy wieloma jednostkami wykonawczymi. Występuje on w sytuacji, gdy liczba wątków możliwych do uruchomienia jest większa niż liczba dostępnych procesorów logicznych. [5]

Nadmierna liczba przełączeń kontekstu prowadzi do istotnego obniżenia wydajności systemu. Wynika to z faktu, że czas procesora jest wtedy zużywany na operacje administracyjne zamiast na wykonywanie właściwych obliczeń. Dodatkowo, jak podają Mogul i Borg [9], częste przełączanie kontekstu negatywnie wpływa na efektywność pamięci podręcznej procesora, powodując utratę lokalności danych i zwiększenie opóźnień dostępu do pamięci.

2.3 Modele współbieżności w środowisku JVM

Środowisko JVM oferuje wiele różnych modeli umożliwiających tworzenie współbieżnych aplikacji. Ta część poświęcona jest omówieniu najpopularniejszych podejść, w tym podstawowego (i chronologicznie pierwszego) modelu, będącego częścią Thread API, modelu opartego o wykorzystanie puli wątków, dostępnego w ramach interfejsu `ExecutorService`, a także modelu reaktywnego.

2.3.1 Klasyczny model współbieżności

Podstawowym modelem współbieżności w środowisku JVM są klasa `Thread` i interfejs `Runnable`, będące częścią pakietu `java.lang` i oferujące podstawowe metody umożliwiające zarządzanie wątkami. Interfejs `Runnable` reprezentuje zadanie przeznaczone do współbieżnego wykonania, natomiast klasa `Thread` odpowiada za jego uruchomienie i wykonanie [10, 11].

Klasa `Thread` jest klasą opakowującą (ang. *wrapper class*) wątki systemu operacyjnego w relacji jeden do jednego. Oznacza to, że każdy obiekt tej klasy odpowiada dokładnie jednemu wątkowi systemowemu, nazywanemu *wątkiem platformowym* (ang. *platform thread*) [12]. Zarządzanie cyklem życia i przełączanie kontekstu realizowane jest przez system operacyjny, a nie przez środowisko JVM. Konsekwencją takiego podejścia jest stosunkowo wysoki koszt tworzenia wątków, zarówno pod względem czasu, jak i zużycia pamięci, co ogranicza ich skalowalność w aplikacjach wymagających obsługi dużej liczby współbieżnych zadań [13].

Jak wskazuje Lea [14], model ten opiera się na współdzielonej pamięci, co oznacza, że wątki mogą uzyskiwać dostęp do tych samych danych. W celu zapewnienia poprawności działania programu konieczne jest stosowanie mechanizmów synchronizacji. Przykładem jest obecne w środowisku JVM słowo kluczowe `synchronized` [15], które umożliwia wzajemne wykluczanie dostępu do sekcji krytycznej.

Pomimo swojej prostoty i bezpośredniego odwzorowania na mechanizmy systemu operacyjnego, model ten posiada istotne ograniczenia. Wysoki koszt tworzenia i zarządzania wątkami oraz podatność na problemy współbieżności, wspomniane w podrozdziale 2.2 Problemy występujące w systemach współbieżnych, sprawiają, że w praktyce często stosuje się bardziej zaawansowane abstrakcje, opisane niżej.

2.3.2 Model współbieżności oparty o pulę wątków

Jednym z rozwiązań wspomnianego wyżej problemu było wprowadzenie w ramach JDK 1.5 interfejsu `ExecutorService` [16]. Został on wprowadzony w celu uproszczenia zarządzania współbieżnością poprzez uniezależnienie definicji zadań od ich wykonania, stanowiąc bardziej elastyczną alternatywę dla bezpośredniego użycia klasy `Thread` i interfejsu `Runnable`.

Interfejs `ExecutorService` w głównej mierze opiera się o wykorzystanie wzorca projektowego, jakim jest *Pula obiektów* (ang. *Object Pool*) [17]. W tym celu interfejs wykorzystuje pulę wątków. Takie podejście pozwala na wielokrotne wykorzystanie tych samych wątków, eliminując konieczność ich częstego tworzenia i niszczenia, co w klasycznym modelu stanowi istotne obciążenie czasowe i pamięciowe [13].

Podejście to jednak nie jest pozbawione ograniczeń. Jednym z nich jest problem odpowiedniego doboru rozmiaru puli wątków.

Określanie rozmiaru puli wątków

Rozmiar puli wątków definiuje się jako liczbę wątków zarządzanych przez mechanizm wykonawczy, które mogą być wykorzystane do równoczesnego wykonywania zadań [18]. Zdaniem Goetza [5], liczba ta powinna być wyznaczana dynamicznie, przykładowo na podstawie liczby dostępnych procesorów logicznych. W środowisku JVM wartość ta jest możliwa do wyznaczenia przy użyciu metody `availableProcessors`, dostępnej w ramach klasy `Runtime` [19].

Goetz wskazuje również, że przy ustalaniu rozmiaru puli wątków należy unikać wartości ekstremalnych. Jeśli pula wątków jest za duża, wątki mogą ze sobą konkurować o ograniczone zasoby procesora i pamięci, co może skutkować większym użyciem pamięci i możliwym wyczerpaniem zasobów, zaś zbyt mała pula wątków może ograniczać przepustowość poprzez niewykorzystanie procesorów logicznych pomimo dostępności zadań do wykonania. Ustalenie właściwego rozmiaru puli wątków zależy między innymi od dostępności zasobów i natury współbieżnie wykonywanych zadań. Goetz podaje, że optymalny rozmiar N_T puli wątków można wyrazić wzorem

$$N_T = N_{\text{CPU}} \cdot U_{\text{CPU}} \cdot \left(1 + \frac{T_W}{T_C}\right),$$

gdzie:

- N_{CPU} oznacza liczbę dostępnych procesorów logicznych,
- U_{CPU} jest wartością docelowego wykorzystania procesora, gdzie $0 \leq U_{\text{CPU}} \leq 1$,
- T_W (ang. *wait time*) reprezentuje średni czas oczekiwania wątku na możliwość kontynuowania pracy, obejmujący między innymi czas oczekiwania na operacje wejścia/wyjścia, dostęp do zasobów współdzielonych lub synchronizację z innymi wątkami,
- T_C (ang. *compute time*) oznacza średni czas, w którym wątek aktywnie wykonuje operacje obliczeniowe, wykorzystując procesor.

Stosunek $\frac{T_W}{T_C}$ odzwierciedla charakter zadań wykonywanych w ramach puli wątków. Niskie wartości tego ilorazu wskazują na zadania obliczeniowe, w których dominującą rolę odgrywa czas przetwarzania (ang. *CPU-bound*), natomiast wysokie wartości są charakterystyczne dla zadań wymagających częstego oczekiwania, takich jak operacje wejścia/wyjścia (ang. *I/O-bound*).

2.3.3 Reaktywny model współbieżności

W odpowiedzi na ograniczenia klasycznych modeli współbieżności opartych na blokujących operacjach i bezpośrednim zarządzaniu wątkami rozwinięto model reaktywny, którego celem jest efektywne przetwarzanie danych w sposób asynchroniczny i nieblokujący. Jednym z kluczowych standardów tego podejścia jest Reactive Streams [20], którego twórcy na oficjalnej stronie projektu podają:

*Reactive Streams to inicjatywa mająca na celu zapewnienie standardu asynchronicznego przetwarzania strumieniowego z wykorzystaniem nieblokującego mechanizmu kontroli przepływu. Obejmuje ona działania skierowane zarówno do środowisk uruchomieniowych [...], jak również dla protokołów sieciowych.*¹

W środowisku JVM model reaktywny obejmuje zestaw wzorców i mechanizmów umożliwiających asynchroniczne, zdarzeniowe oraz nieblokujące przetwarzanie danych z wykorzystaniem strumieni. W praktyce jego realizacja opiera się na specyfikacji Reactive Streams oraz bibliotekach takich jak Project Reactor [21], RxJava [22] czy Akka Streams [23], które dostarczają narzędzi do budowy skalowalnych aplikacji współbieżnych.

Podstawowe pojęcia związane z programowaniem reaktywnym

Jak podają Dokuka i Lozynski [24], przekazywanie informacji w modelu reaktywnym odbywa się w sposób *asynchroniczny*, tj. operacje nie są wykonywane w sposób blokujący, a ich wynik nie jest dostępny natychmiast po wywołaniu, jak w modelu klasycznym. Taki system oparty jest o przekazywanie *komunikatów* (ang. *messages*) za pomocą *strumieni danych* (ang. *data streams*). Dane te, w zależności od roli, mogą być wysyłane i odbierane przez specjalnie zdefiniowane komponenty. Dokumentacja Reactive Streams [20] wskazuje, że w reaktywnym modelu współbieżności wyróżnia się cztery podstawowe role:

- *producent* (ang. *producer, publisher*), który publikuje komunikaty odbierane przez odbiorcę,
- *odbiorca* (ang. *receiver, subscriber*), który otrzymuje komunikaty od producenta,
- *procesor* (ang. *processor*), który pełni rolę zarówno producenta, jak i odbiorcy,
- *subskrypcja* (ang. *subscription*), która jest mechanizmem zarządzającym relacją między producentem a odbiorcą.

¹Tłumaczenie własne na podstawie [20].

W środowisku JVM role te są zaimplementowane odpowiednio w ramach interfejsów `Publisher<T>`, `Subscriber<T>`, `Processor<T,R>` i `Subscription`, będących składowymi klasy `java.util.concurrent.Flow` [25].

Model reaktywny jest silnie powiązany ze wzorcem projektowym *Obserwator* (ang. *Observer*) [26]. W obu przypadkach komunikacja pomiędzy komponentami opiera się na mechanizmie publikowania i subskrypcji zdarzeń. Kluczową różnicą jest jednak obecność mechanizmu *backpressure*, który umożliwia odbiorcy kontrolę tempa napływu danych, czego klasyczna definicja tego wzorca nie zapewnia.

Cechy modelu reaktywnego

Autorzy dokumentu *The Reactive Manifesto* [27], stanowiącego zbiór zasad i wytycznych dotyczących projektowania systemów reaktywnych, podają następujące cechy systemu opartego o reaktywny model przetwarzania danych:

- *responsywność* (zapewnienie szybkich i spójnych czasów reakcji celem zapewnienia jak najwyższej jakości usług),
- *odporność* (zapewnienie responsywności systemu w przypadku wystąpienia awarii poprzez hermetyzację i izolację przyczyny awarii od reszty systemu, a także replikację i delegowanie przetwarzania do innych, zewnętrznych komponentów),
- *elastyczność* (zapewnienie responsywności przy zmiennym obciążeniu systemu poprzez zastosowanie różnych algorytmów skalowania),
- *zarządzanie za pomocą komunikatów* (ang. *message-driven*; zapewnienie komunikacji między komponentami poprzez asynchroniczne przekazywanie komunikatów).

Wymienione właściwości stanowią zestaw ogólnych zasad projektowych, które definiują sposób budowy systemów reaktywnych. Ich zastosowanie wpływa na sposób organizacji komunikacji między komponentami oraz zarządzania obciążeniem i błędami w systemach współbieżnych [27].

Mechanizm kontroli przepływu danych (*backpressure*)

Jednym z kluczowych mechanizmów wykorzystywanych w reaktywnym modelu współbieżności jest *mechanizm kontroli przepływu danych*, częściej w literaturze określany mianem *backpressure*. Jak podaje Davis [28], mechanizm ten umożliwia konsumentowi ograniczenie tempa napływu danych od producenta. Jego celem jest zapobieganie przeciążeniu odbiorcy poprzez dostosowanie szybkości generowania danych do możliwości ich przetwarzania. W praktyce realizowane jest to poprzez

jawne sygnalizowanie przez konsumenta zapotrzebowania na określoną liczbę elementów, co pozwala producentowi kontrolować tempo emisji danych i dostosować je do aktualnych możliwości odbiorcy. W środowisku JVM mechanizm ten realizowany jest między innymi w ramach specyfikacji Reactive Streams poprzez metodę `request(long n)` interfejsu `Subscription`, za pomocą której konsument określa liczbę elementów, jakie jest gotowy przetworzyć [25].

Rozdział 3

Mechanizmy współbieżności oferowane przez Project Loom

Trzeci rozdział pracy poświęcony jest omówieniu najważniejszych mechanizmów dostępnych w ramach Project Loom – inicjatywy usprawniającej tworzenie aplikacji współbieżnych w środowisku JVM. Rozdział prezentuje historię powstawania projektu, a także omawia najważniejsze mechanizmy, które on oferuje, tj. mechanizm wirtualnych wątków (ang. *virtual threads*), mechanizm współbieżności strukturalnej (ang. *structured concurrency*) oraz mechanizm wartości zakresowych (ang. *scoped values*).

3.1 Wprowadzenie do Project Loom

Project Loom jest inicjatywą zapoczątkowaną pod koniec 2017 roku przez OpenJDK. Wyróżnia się jego trzy główne składowe:

- mechanizm *wirtualnych wątków* (ang. *virtual threads*), będący alternatywą dla klasycznego mechanizmu wątków platformowych i zdefiniowany w dokumencie JEP¹ 444 [30],
- mechanizm *współbieżności strukturalnej* (ang. *structured concurrency*), umożliwiający uporządkowanie pracy z wątkami poprzez dodanie *zakresów zadań* (ang. *task scopes*) i strategii zachowań w przypadku wystąpienia błędu lub przerwania działania wątku, opisany w dokumencie JEP 525 [31],
- mechanizm *wartości zakresowych* (ang. *scoped values*), będący alternatywą dla klasy `ThreadLocal` w systemach współbieżnych i opisany w dokumencie JEP 506 [32].

¹JEP (*JDK Enhancement Proposal*) to formalny dokument opisujący proponowane zmiany, nowe funkcjonalności lub usprawnienia platformy JDK. [29]

Początkowo ideą projektu było wprowadzenie lekkich wątków użytkownika (ang. *user-mode threads*) oraz uproszczenie programowania współbieżnego, szczególnie w aplikacjach, których dużą częścią są blokujące zadania, takich jak serwery i systemy sieciowe [33]. Wraz z upływem czasu zakres projektu został rozszerzony o mechanizmy zwiększające bezpieczeństwo aplikacji współbieżnych i ustrukturyzowanie pracy z wątkami. Project Loom wciąż ewoluuje, głównie w kontekście zmian w API poszczególnych mechanizmów, lecz większość z nich jest integralną częścią JDK, dodanych w ramach oficjalnych wydań.

3.2 Wirtualne wątki (*Virtual Threads*)

Mechanizm wirtualnych wątków, we wczesnych fazach projektu określanych mianem *fibers* [33], to chronologicznie pierwsza z trzech głównych składowych projektu Loom. Po raz pierwszy mechanizm ten pojawił się w środowisku JDK w wersji 19 w ramach wersji poglądowej (ang. *preview*), natomiast jego stabilna wersja została udostępniona w ramach JDK 21. Stanowi on alternatywę dla wątków platformowych, opisanych w sekcji 2.3.1 Klasyczny model współbieżności.

W celu wyjaśnienia potrzeby wprowadzenia mechanizmu wirtualnych wątków do środowiska JVM, dokument JEP 444 [30] odwołuje się do prawa Little’a [34], które wiąże ze sobą latencję, współbieżność i przepustowość w kontekście skalowalności aplikacji współbieżnych:

Dla danego czasu przetwarzania żądania (tj. latencji) liczba żądań obsługiwanych jednocześnie przez aplikację (tj. współbieżność) musi rosnąć proporcjonalnie do tempa napływu żądań (tj. przepustowości).²

Z zależności tej wynika, że zwiększenie przepustowości systemu wymaga zwiększenia liczby jednocześnie obsługiwanych zadań. W klasycznym modelu *thread-per-request*³ prowadzi to jednak do konieczności tworzenia dużej liczby wątków platformowych, co wiąże się z istotnym narzutem pamięciowym oraz kosztami zarządzania, takimi jak przełączanie kontekstu, o czym wspomniano w rozdziale drugim (patrz sekcje 2.2.3 i 2.3.1). W konsekwencji ogranicza to skalowalność aplikacji oraz utrudnia efektywne wykorzystanie dostępnych zasobów sprzętowych.

Alternatywą dla tego podejścia jest model asynchroniczny, w którym liczba wątków jest ograniczana poprzez ich współdzielenie pomiędzy wieloma zadaniami. Rozwiązanie to pozwala zwiększyć skalowalność, jednak kosztem znacznego wzrostu

²Tłumaczenie własne na podstawie [30].

³Paradygmat *thread-per-request* zakłada, że każde przychodzące żądanie jest obsługiwane przez dedykowany wątek. [5]

złożoności kodu, wymagając dekompozycji logiki na mniejsze etapy oraz, w przypadku reaktywnego modelu współbieżności, zrezygnowania z imperatywnego (tj. sekwencyjnego) paradygmatu pisania kodu, co stanowi dodatkowy wkład programisty w utrzymywanie kodu. W rezultacie pojawiła się potrzeba rozwiązania, które łączyłoby prostotę modelu synchronicznego z wydajnością podejścia asynchronicznego, co stanowi bezpośrednią motywację dla wprowadzenia wątków wirtualnych. Stanowią one próbę pogodzenia tych dwóch podejść poprzez umożliwienie tworzenia bardzo dużej liczby lekkich wątków przy zachowaniu prostego, sekwencyjnego modelu programowania. Dzięki temu możliwe jest zwiększenie poziomu współbieżności bez ponoszenia kosztów charakterystycznych dla wątków platformowych oraz bez konieczności stosowania złożonych mechanizmów asynchronicznych.

3.2.1 Zasada działania i cykl życia wirtualnych wątków

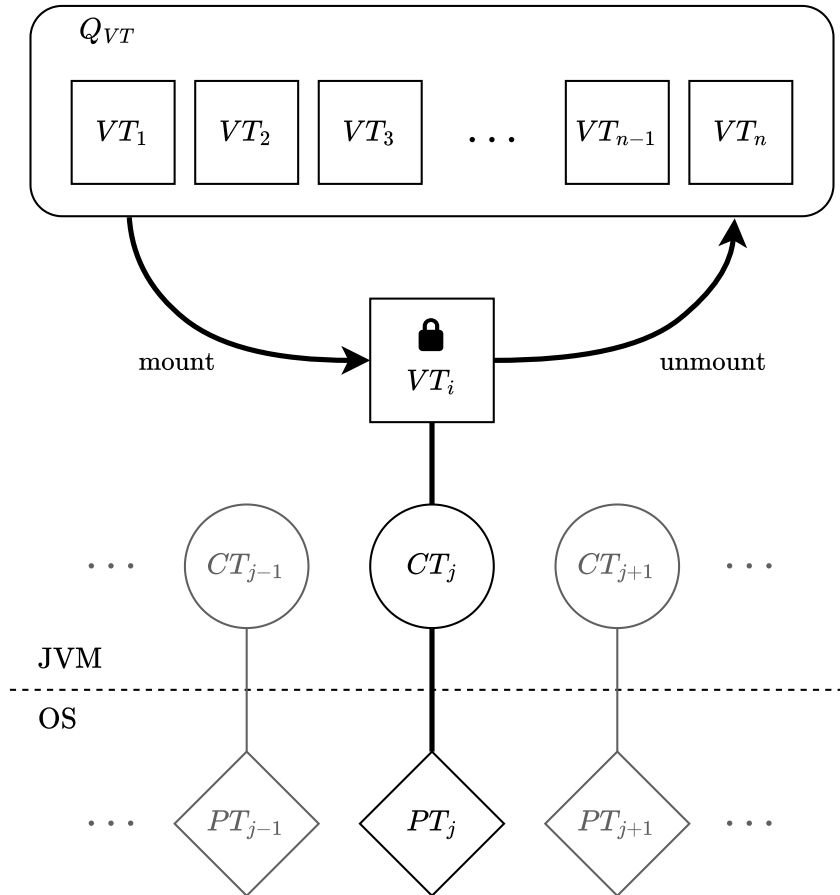
Każdorazowo, gdy tworzony jest wirtualny wątek, zostaje on dodany do *kolejki wirtualnych wątków*, będącą kolejką typu FIFO⁴. Wirtualny wątek może wykonać przydzielone mu zadanie, gdy zostanie przypisany do *wątku nośnego* (ang. *carrier thread*). Reprezentuje on wątek platformowy po stronie platformy JVM [13]. Każdy wątek nośny ma przypisany odpowiadający mu wątek platformowy i domyślnie ich liczba jest równa liczbie dostępnych (fizycznych) procesorów logicznych [36]. W odróżnieniu od wątków platformowych i nośnych, liczba wątków wirtualnych nie jest zależna od liczby dostępnych procesorów logicznych, a w większości przypadków ją przekracza. Wynika to ze sposobu zarządzania cyklem ich życia, który został przedstawiony na rysunku 3.1.

Korzystając z oznaczeń zawartych na rysunku 3.1, cykl życia wirtualnych wątków w środowisku JVM w kontekście jednego wątku nośnego można opisać następująco:

1. Każdy nowy wirtualny wątek VT zostaje dodany na koniec kolejki wirtualnych wątków Q_{VT} .
2. Planista (ang. *scheduler*) wątków wirtualnych, obecny w środowisku JVM, bierze pierwszy wolny wątek wirtualny z kolejki i przypisuje go (ang. *mount*) do wolnego wątku nośnego CT_j .
3. Wątek nośny CT_j , odpowiadający wątkowi platformowemu PT_j , wykonuje kod przypisany do przydzielonego mu wątku wirtualnego VT_i .

⁴Kolejka typu FIFO (ang. *First-In, First-Out*) jest strukturą danych, w której elementy są usuwane w tej samej kolejności, w jakiej zostały dodane, tj. pierwszy dodany element jest obsługiwany jako pierwszy. [35]

4. Gdy wątek wirtualny VT_i wykonuje operację blokującą (stan reprezentowany przez symbol ) JVM zapisuje jego aktualny stan, dodaje go na koniec kolejki Q_{VT} , jednocześnie zwalniając (ang. *unmount*) wątek nośny CT_j .
5. JVM przypisuje kolejny wątek wirtualny (tj. pierwszy w kolejce Q_{VT}) do wątku nośnego CT_j .



Rysunek 3.1: Cykl życia wirtualnych wątków w środowisku JVM
(źródło: opracowanie własne na podstawie [13])

Kluczową różnicą względem wątków platformowych w powyższym opisie jest punkt 4. Podczas wykonywania operacji blokującej wątek nośny nie wstrzymuje dalszego wykonywania operacji, lecz zapisuje aktualny stan przypisanego do niego wątku wirtualnego, dodaje go do kolejki Q_{VT} i wykonuje operacje przypisane do następnego w kolejce wątku wirtualnego. Analogiczna sytuacja z wykorzystaniem wątków platformowych skutkowałaby zablokowaniem tego wątku na czas wykonywania operacji blokującej, tj. wątek ten nie mógłby w tym czasie wykonać żadnej innej operacji. Taki sposób zarządzania wątkami pozwala efektywniej wykorzystać czas bezczynności wątku wywoływany przez operacje blokujące, a tym samym pozwala zwiększyć liczbę żądań obsługiwanych jednocześnie przez aplikację. [13]

3.2.2 Pule wirtualnych wątków

Jak wskazują autorzy dokumentu JEP 444 [30], wirtualne wątki nie powinny być wykorzystywane do tworzenia puli wątków. Wynika to z faktu, że w ogólności wzorzec puli obiektów jest przeznaczony do współdzielenia kosztownych zasobów. W przeciwieństwie do wątków platformowych, koszt tworzenia i utrzymywania wątków wirtualnych jest niski, co jest głównym powodem ku temu, aby nie tworzyć puli wirtualnych wątków. Ta cecha umożliwia stosowanie wcześniej wspomnianego podejścia thread-per-request, a w przypadku potrzeby zdefiniowania maksymalnej liczby współbieżnie wykonywanych zadań, do czego byłoby można zastosować pulę wątków wirtualnych o ustalonym maksymalnym rozmiarze, autorzy dokumentu rekomendują wykorzystanie semaforów [37].

3.3 Współbieżność strukturalna (*Structured Concurrency*)

Współbieżność strukturalna (ang. *structured concurrency*) jest kolejną z trzech głównych składowych projektu Loom. Mechanizm ten pojawił się po raz pierwszy wraz z mechanizmem wirtualnych wątków w JDK w wersji 19 w ramach wersji pogładowej, lecz nie został on do tej pory oficjalnie wydany.⁵ Współbieżność strukturalna dostarcza mechanizmy umożliwiające uporządkowanie relacji pomiędzy zadaniami wykonywanymi równolegle poprzez odwzorowanie ich w strukturze programu. W odróżnieniu od innych mechanizmów współbieżności, w których zadania mogą być uruchamiane i kończone niezależnie od siebie, mechanizm ten traktuje grupę powiązanych zadań jako jedną jednostkę obliczeniową, umożliwiając ich wspólne zarządzanie, synchronizację oraz obsługę błędów. [31]

Jak podają Veen i Vlijmincx [13], podejście to opiera się na założeniu, że jeśli zadanie dzieli się na podzadania wykonywane współbieżnie, to wszystkie powinny zakończyć się w tym samym miejscu w kodzie, w którym zostały utworzone. Dzięki temu cykl życia zadań jest ściśle powiązany ze strukturą składniową programu, co upraszcza analizę, zwiększa czytelność oraz poprawia niezawodność aplikacji współbieżnych.

⁵Zgodnie ze stanem na kwiecień 2026 roku najnowsza wersja mechanizmu współbieżności strukturalnej została udostępniona w wersji *preview*, co oznacza, że jest dostępna w standardowym JDK, lecz jej składnia lub semantyka mogą ulec zmianie przed ostatecznym włączeniem do specyfikacji platformy Java.

3.3.1 Cechy współbieżności strukturalnej

W klasycznym podejściu do współbieżności, określanym jako *współbieżność niestukturalna* (ang. *unstructured concurrency*), zadania są uruchamiane niezależnie od struktury programu, przykładowo przy użyciu interfejsu `ExecutorService`. W takim modelu cykl życia podzadań nie jest bezpośrednio powiązany z zadaniem nadrzędnym, co prowadzi do sytuacji, w której podzadania mogą zakończyć się później niż zadanie, które je utworzyło, lub kontynuować działanie mimo wystąpienia błędu w innym fragmencie programu.

W przeciwieństwie do tego, współbieżność strukturalna narzuca hierarchiczną organizację zadań, w której każde podzadanie jest jednoznacznie powiązane z zadaniem nadrzędnym. Wszystkie podzadania muszą zakończyć się przed zakończeniem zadania nadrzędnego, a ich cykl życia jest ograniczony do określonego bloku kodu. Takie podejście umożliwia traktowanie grupy zadań jako jednej jednostki, co upraszcza zarządzanie ich wykonaniem oraz obsługę błędów. [13]

Autorzy dokumentu JEP 525 [31] wskazują, że mechanizm współbieżności strukturalnej oferuje automatyczną propagację błędów oraz anulowanie powiązanych podzadań. W przypadku niepowodzenia jednego z podzadań możliwe jest anulowanie pozostałych, co zapobiega niekontrolowanemu zużyciu zasobów i upraszcza logikę obsługi błędów.

3.3.2 Struktura i działanie zakresów zadań

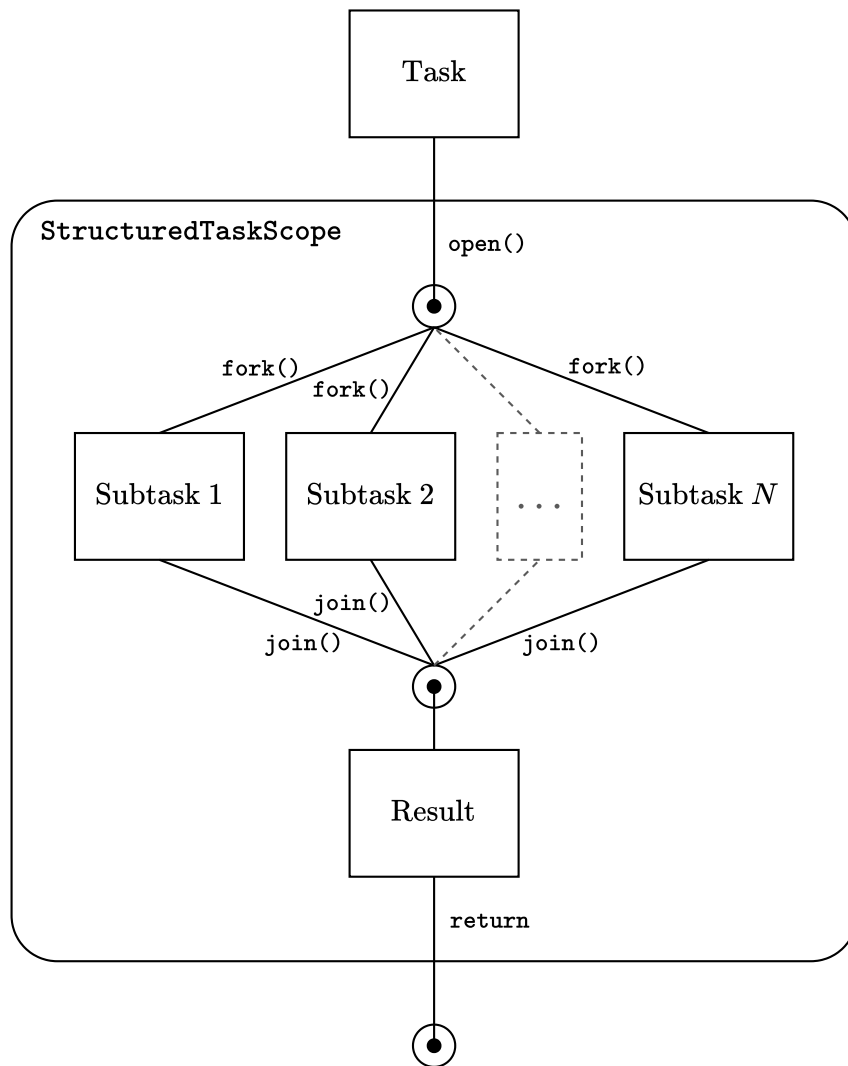
Podstawowym elementem API współbieżności strukturalnej w środowisku JVM jest klasa `StructuredTaskScope` [38], która umożliwia definiowanie zakresu wykonywania współbieżnych podzadań. Zakres ten wyznacza kontekst, w którym tworzone są podzadania oraz w którym następuje ich synchronizacja i obsługa wyników.

Dokument JEP 525 wskazuje, że ogólny przepływ pracy kodu wykorzystującego klasę `StructuredTaskScope` można przedstawić następująco:

1. Utworzenie nowego zakresu zadań (ang. *task scope*) poprzez wywołanie jednej z metod `open`. Wątek, który otwiera zakres, staje się jego właścicielem.
2. Uruchomienie podzadań (ang. *subtasks*) w ramach zakresu przy użyciu metody `fork`.
3. Synchronizacja wszystkich podzadań zakresu jako całości za pomocą metody `join`, która może zgłosić wyjątek.
4. Przetworzenie uzyskanych wyników.

5. Zamknięcie zakresu (najczęściej w sposób niejawny przy użyciu konstrukcji `try-with-resources`⁶), co powoduje anulowanie zakresu (jeżeli nie został wcześniej anulowany), a tym samym anulowanie wszystkich pozostałych podzadań oraz oczekiwanie na ich zakończenie.

Rysunek 3.2 przedstawia zgodny z powyższym opisem schemat działania współbieżności strukturalnej w ramach klasy `StructuredTaskScope`.



Rysunek 3.2: Schemat działania zakresu zadań w ramach klasy `StructuredTaskScope` w środowisku JVM
(źródło: opracowanie własne na podstawie [13])

⁶Konstrukcja `try-with-resources` umożliwia automatyczne zamykanie zasobów po zakończeniu ich użycia. [39]

3.3.3 Strategie synchronizacji podzadań

Mechanizm współbieżności strukturalnej umożliwia definiowanie różnych strategii synchronizacji podzadań za pomocą interfejsu `Joiner` [40]. Odpowiada on za określenie sposobu zakończenia całego zakresu oraz przetwarzania wyników podzadań.

Interfejs posiada statyczne metody definiujące cztery podstawowe strategie synchronizacji podzadań:

- `allSuccessfulOrThrow()` – strategia wymagająca poprawnego zakończenia wszystkich podzadań; w przypadku błędu któregośkolwiek z nich zakres zostaje anulowany, a metoda `join` zgłasza wyjątek,
- `anySuccessfulResultOrThrow()` – strategia zwracająca wynik pierwszego poprawnie zakończonego podzadania; jeżeli wszystkie podzadania zakończą się błędem, zgłaszany jest wyjątek,
- `awaitAllSuccessfulOrThrow()` – strategia oczekująca na zakończenie wszystkich podzadań, przy czym wymaga ich poprawnego wykonania; w przypadku błędu któregośkolwiek z nich zakres zostaje anulowany, a metoda `join` zgłasza wyjątek (strategia ta nie zwraca wyników, a jedynie sygnalizuje powodzenie lub porażkę całej operacji),
- `awaitAll()` – strategia oczekująca na zakończenie wszystkich podzadań niezależnie od ich wyniku; nie powoduje anulowania zakresu ani zgłoszenia wyjątku w przypadku błędów, co czyni ją użyteczną w scenariuszach opartych na efektach ubocznych zamiast zwracanych wartościach.

Wybór odpowiedniej strategii synchronizacji zależy od charakteru problemu oraz oczekiwanego sposobu obsługi błędów i wyników podzadań. Interfejs `Joiner` umożliwia również tworzenie własnych strategii synchronizacji. Odbyna się to poprzez implementację interfejsu w klasie reprezentującej własną strategię oraz odpowiednie nadpisanie metod `onFork`, `onComplete`, `onTimeout` i `result`. [31, 40]

3.4 Wartości zakresowe (*Scoped Values*)

Wartości zakresowe (ang. *scoped values*) stanowią mechanizm projektu Loom umożliwiający współdzielenie niezmiennych danych pomiędzy metodą a jej wywołaniami w obrębie jednego wątku, a także pomiędzy wątkiem nadrzędnym i jego wątkami potomnymi. Ich głównym celem jest zastąpienie mechanizmu oferowanego przez klasę `ThreadLocal` [41], który umożliwia przechowywanie danych specyficznych dla danego wątku, zapewniając każdemu wątkowi własną, niezależną

kopię zmiennej. Jak podaje dokument JEP 506, mechanizm wartości zakresowych charakteryzuje się niższą złożonością czasową i pamięciową niż mechanizm dostępny w ramach klasy `ThreadLocal`, zwłaszcza w połączeniu z innymi mechanizmami oferowanymi przez Project Loom [32]. Mechanizm wartości zakresowych po raz pierwszy pojawił się w JDK w wersji 20 w wersji eksperymentalnej, a oficjalnie został włączony do środowiska JVM w ramach wydania JDK 25.

3.4.1 Opis mechanizmu `ThreadLocal`

Jak podają Veen i Vlijmincx [13], klasa `ThreadLocal` stanowi mechanizm umożliwiający przechowywanie danych specyficznych dla danego wątku. Zmienne typu `ThreadLocal` są powiązane z konkretnym wątkiem, dzięki czemu każdy wątek posiada własną, niezależną wartość tej zmiennej i może uzyskać dostęp wyłącznie do swojej kopii.

Mechanizm `ThreadLocal` zapewnia wbudowaną izolację danych pomiędzy wątkami, eliminując konieczność stosowania synchronizacji i upraszczając kod. Umożliwia również wygodne przechowywanie informacji kontekstowych, takich jak dane użytkownika czy kontekst transakcji, bez potrzeby przekazywania ich jako parametrów. Dodatkowo pozwala na definiowanie konfiguracji specyficznej dla wątku. [13, 41]

Pomimo swojej użyteczności, klasa `ThreadLocal` posiada istotne ograniczenia. Jednym z nich jest fakt, że wartości przechowywane wewnątrz obiektu tej klasy są mutowalne, tj. mogą być modyfikowane w dowolnym miejscu kodu, co utrudnia analizę przepływu danych oraz znacząco komplikuje proces debugowania. Ponadto brak jawnego zarządzania cyklem życia wartości może prowadzić do wycieków pamięci, szczególnie w przypadku ponownego wykorzystywania wątków w pulach, gdzie nieusunięte dane pozostają powiązane z ponownie używanymi wątkami. Dodatkowo w środowiskach współbieżnych, zwłaszcza przy wykorzystaniu wątków wirtualnych, mechanizm ten może powodować nadmierne zużycie pamięci, wynikające z konieczności kopiowania wartości do wątków potomnych, co w konsekwencji może prowadzić do problemów ze skalowalnością. [13, 41]

3.4.2 Zasada działania wartości zakresowych

Mechanizm wartości zakresowych stanowi alternatywę dla klasy `ThreadLocal`, wprowadzając odmienny model współdzielenia danych. W przeciwieństwie do `ThreadLocal`, gdzie wartości są powiązane z całym cyklem życia wątku, wartości zakresowe obowiązują wyłącznie w określonym zakresie wykonania programu. Oznacza to, że ich widoczność jest ograniczona do dynamicznego kontekstu wywołań metod, w którym zostały zdefiniowane, a po jego zakończeniu są automatycznie usuwane.

Takie podejście eliminuje konieczność ręcznego zarządzania cyklem życia danych oraz ogranicza ryzyko błędów wynikających z niekontrolowanego współdzielenia stanu. [32]

Podstawą działania mechanizmu jest jawne powiązanie wartości z określonym zakresem wykonania (ang. *binding*). Realizowane jest ono poprzez metody klasy `ScopedValue` [42], które umożliwiają utworzenie kontekstu, w ramach którego dana wartość jest dostępna. Wartość ta może być następnie odczytywana przez wszystkie metody wywoływane bezpośrednio lub pośrednio w obrębie tego zakresu. Kluczową cechą tego podejścia jest dynamiczny charakter zakresu, tj. jego granice są wyznaczone przez przebieg wykonania programu, a nie przez strukturę składniową kodu. [32, 13]

Główny mechanizm wartości zakresowych opiera się na kilku podstawowych metodach klasy `ScopedValue` i klas wewnętrznych. Metody `where().run()` oraz `where().call()` umożliwiają powiązanie wartości z danym zakresem oraz wykonanie kodu w jego obrębie. Dostęp do wartości realizowany jest za pomocą metody `get()`. Dodatkowo klasa oferuje metodę `isBound()`, pozwalającą sprawdzić, czy dana wartość jest aktualnie powiązana z bieżącym kontekstem wykonania [42].

W odróżnieniu od mechanizmu oferowanego w ramach klasy `ThreadLocal`, wartości zakresowe są niezmiennie w obrębie danego zakresu, co oznacza, że po ich przypisaniu nie mogą być modyfikowane. Możliwe jest jednak ich ponowne powiązanie w zagnieżdżonym zakresie, co prowadzi do tymczasowego przesłonięcia wcześniejszej wartości. Takie podejście zapewnia większą przewidywalność działania programu oraz ułatwia analizę przepływu danych, szczególnie w aplikacjach współbieżnych. [42]

Rozdział 4

Przegląd literatury i opis stanu badań nad Project Loom

Czwarty rozdział pracy zawiera opis stanu badań i wnioski wynikające z przeglądu literatury we względnie nowej dziedzinie, jaką jest Project Loom, kładąc szczególny nacisk na badania opisujące zastosowania tych mechanizmów. Rozdział ten definiuje także lukę badawczą na podstawie dokonanego przeglądu oraz określa kierunek przeprowadzonych badań.

4.1 Opis i charakterystyka badań nad Project Loom

Mimo że Project Loom i badania wykorzystujące jego mechanizmy są obszarem stosunkowo nowym, istnieją już prace empiryczne, analizy porównawcze i prace dyplomowe badające ich zalety, ograniczenia i praktyczne efekty. Wyróżnia się trzy kategorie, na które można podzielić publikacje opisujące lub badające tematykę Project Loom:

- prace akademickie, skupiające się w dużej mierze na testach empirycznych, porównując mechanizmy oferowane przez Project Loom z innymi modelami współbieżności, w szczególności pod kątem wysokiego obciążenia mocy obliczeniowej procesora (ang. *CPU-bound*) i wysokiego obciążenia wejścia i wyjścia (ang. *I/O-bound*),
- dokumentacja techniczna i dokumenty JEP (JDK Enhancement Proposal), stanowiące fundament teoretyczny i szczegółowo opisujące potrzebę wprowadzenia mechanizmów będących częścią Project Loom, ich zastosowania i dogłębne aspekty techniczne,
- inne teksty dotyczące mechanizmów oferowanych przez Project Loom, w tym nietechniczne artykuły oraz wpisy na blogach i forach.

4.2 Przegląd wybranych pozycji

4.2.1 Prace akademickie

Ponnampalam w pracy „Evaluation of Virtual Threads and RxJava: A performance and code complexity analysis in a real-time clearing system” [43] porównuje wątki wirtualne z biblioteką reaktywną RxJava w realnym systemie transakcyjnym pod względem wydajności (mierząc takie parametry, jak latencja i przepustowość), zużycia pamięci i procesora oraz złożoności kodu. Wyniki uzyskane przez autora wskazują, że wirtualne wątki przewyższają mechanizmy reaktywne w scenariuszach z dużą liczbą blokujących operacji i przy wysokiej współbieżności, zużywając mniej zasobów i upraszczając kod. Ponnampalam zaznacza jednak, że w wybranych, wysoce zoptymalizowanych przypadkach reaktywnych, zastosowanie biblioteki RxJava może być porównywalnie wydajne jak zastosowanie wirtualnych wątków. .

Podobne wnioski wysnuł Pandita w pracy „Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads” [44], w ramach której badana była wydajność i wykorzystanie zasobów przez wątki wirtualne i platformowe w obciążeniach intensywnie wykorzystujących procesor i przy wysokim obciążeniu wejścia i wyjścia. Autor wskazał, że dla obciążeń I/O-bound wirtualne wątki wykazują większą skalowalność i obsługują większe obciążenie z przy jednoczesnym mniejszym zużyciu pamięci i niższej latencji. Dodał również, że dla obciążeń CPU-bound oba modele zachowują się podobnie. Ponadto, wysoka współbieżność przy zastosowaniu wirtualnych wątków zwiększa przepustowość nawet o około 60%, a latencja spada o około 28% w porównaniu do wątków platformowych.

Praca Gustafssona i Perssona zatytułowana „Comparison of Concurrency Technologies in Java” [45] dotyczy empirycznego porównania trzech podejść do współbieżności w Javie w kontekście aplikacji serwerowych o dużym obciążeniu. Autorzy koncentrują się na trzech technologiach: wątkach platformowych, wirtualnych i modelu reaktywnym. Autorzy zaimplementowali w tym celu serwer HTTP, inspirowany benchmarkiem Fortune (Techempower) z dodatkowymi opóźnieniami, aby wymusić różne wzorce obciążenia. Autorzy pracy doszli do następujących wniosków w kontekście wirtualnych wątków:

- wirtualne wątki przewyższają wątki platformowe w przepustowości i latencji w wielu scenariuszach aż do pewnego progu obciążenia,
- wirtualne wątki bardziej obciążają zużycie procesora i pamięci niż model reaktywny, szczególnie przy krótkich operacjach blokujących,

- w testach CPU-bound wątki platformowe były w niektórych przypadkach szybsze od wirtualnych wątków, ale zużywały mniej CPU tylko w konkretnych ustawieniach.

Ponadto autorzy zauważyli, że wątki wirtualne mogą szybciej osiągać wysoki poziom wykorzystania pamięci, co jest zgodne z efektem widocznym w praktyce przy intensywnym tworzeniu i obsłudze wątków, szczególnie przy operacjach wejścia i wyjścia. Wskazują również, że różnice w wydajności wirtualnych wątków i modelu reaktywnego mogą wynikać z zachowania garbage collector, który intensywniej działa przy dużej liczbie krótkotrwałych obiektów.

Publikacje innych autorów [13, 46, 47, 48, 49] w dużym stopniu pokrywają lub uzupełniają tematykę i zakres badań przytoczonych wyżej prac. Przegląd literatury nie wykazał istnienia prac pokrywających inne zagadnienia związane z mechanizmami Project Loom niż przytoczone wyżej.

4.2.2 Dokumentacja techniczna

Dokumentacja techniczna związana z mechanizmami oferowanymi przez Project Loom ogranicza się do dokumentów JEP oraz do oficjalnej dokumentacji języka Java. Dokumenty „JEP 444: Virtual Threads” [30], „JEP 525: Structured Concurrency (Sixth Preview)” [31] i „JEP 506: Scoped Values” [32] szczegółowo opisują techniczne aspekty wskazanych mechanizmów współbieżności i potrzebę ich wprowadzenia. Wskazują również podstawowe zastosowania i ograniczenia wprowadzonych mechanizmów.

4.3 Zdefiniowanie luki badawczej i kierunku własnych badań

Przegląd publikacji związanych z zastosowaniem mechanizmów oferowanych przez Project Loom wykazał brak pokrycia innych aspektów ich wykorzystania niż w kontekście wysokiego obciążenia mocy obliczeniowej procesora i wysokiego obciążenia wejścia i wyjścia. Zauważono również, że wszystkie publikacje badają zastosowanie wirtualnych wątków, bez uwzględnienia pozostałych mechanizmów, tj. współbieżności strukturalnej i wartości zakresowych. Nie znaleziono innych publikacji badających integrację mechanizmów dostępnych w ramach Project Loom z istniejącymi mechanizmami dostępnymi w środowisku JVM niż przytoczone, głównie opartymi o operacje blokujące lub programowanie asynchroniczne.

W ramach niniejszej pracy zdecydowano się pokryć lukę badawczą poprzez implementację dwóch scenariuszy:

- zastosowania mechanizmu wirtualnych wątków w kontekście integracji z bazą danych z wykorzystaniem sterownika blokującego (JDBC) i reaktywnego (R2DBC),
- zastosowania mechanizmu współbieżności strukturalnej w kontekście agregacji wyników opartej na mechanizmie kworum.

Szczegółowy opis implementacji obu scenariuszy zawarty jest w rozdziale piątym.

Rozdział 5

Metodyka badań

Piąty rozdział prezentuje metodykę badań przeprowadzonych w ramach pracy, w tym cel i zakres badań oraz szczegółowy opis dwóch scenariuszy testowych: scenariusza $\mathcal{A}$ polegającego na integracji z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC) oraz scenariusza $\mathcal{B}$ polegającego na agregacji wyników opartej na mechanizmie kworum, prezentując dla każdego z nich przyjęte założenia i opisując ich implementację i kontekście wybranych modeli współbieżności. Rozdział definiuje również hipotezy badawcze, określone na podstawie pytań badawczych, zdefiniowanych scenariuszy testowych i przeanalizowanej literatury.

5.1 Cel i zakres badań

Głównym celem badań jest analiza porównawcza i integracyjna mechanizmów oferowanych w ramach Project Loom, w szczególności mechanizmu wirtualnych wątków i mechanizmu współbieżności strukturalnej, z istniejącymi modelami współbieżności w kontekście dwóch scenariuszy: integracji z bazą danych przy wykorzystaniu różnych sterowników umożliwiających połączenie z bazą (dalej określany jako scenariusz $\mathcal{A}$) oraz agregacji wyników opartej na mechanizmie kworum (dalej określany jako scenariusz $\mathcal{B}$). Badania mają na celu zarówno sprawdzenie wydajności tych mechanizmów w porównaniu do istniejących modeli współbieżności, jak i analizę kodu powstałego w ramach implementacji scenariuszy pod kątem złożoności implementacyjnej i wbudowanej odporności na błędy w użytych modelach współbieżności.

5.2 Szczegółowy opis scenariuszy testowych

5.2.1 Scenariusz $\mathcal{A}$: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)

Scenariusz $\mathcal{A}$ ma na celu zbadanie wpływu doboru modelu współbieżności na integrację z bazą danych przy wykorzystaniu różnych sterowników odpowiedzialnych za komunikację z bazami danych, ze szczególnym uwzględnieniem modelu współbieżności opartego o mechanizmy oferowane przez Project Loom.

W środowisku JVM wyróżnia się dwa główne sterowniki:

- JDBC (ang. *Java Database Connectivity*), oferujący standardowe API, w którym operacje są wykonywane w sposób blokujący [50],
- R2DBC (ang. *Reactive Relational Database Connectivity*), oferujący API zaprojektowane do podejścia reaktywnego, w którym operacje na bazie są nieblokujące i oparte na strumieniach danych [51].

W ramach eksperymentu skonstruowano trzy warianty reprezentujące różne modele współbieżności:

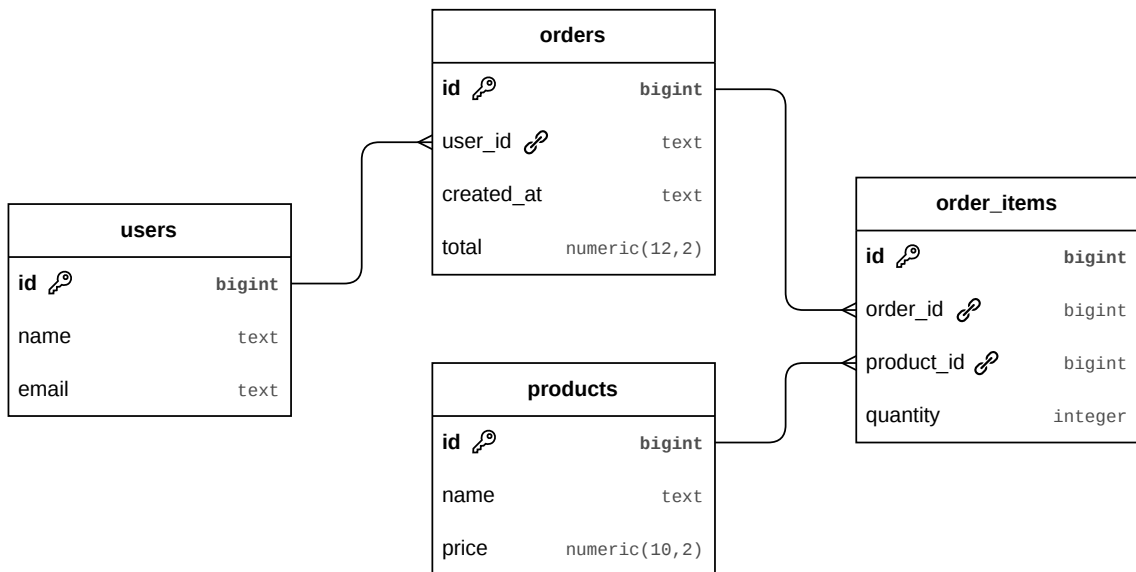
- JDBC+PT, wykorzystujący sterownik JDBC przy wykorzystaniu wątków platformowych,
- R2DBC, wykorzystujący sterownik R2DBC oraz reaktywny model współbieżności,
- JDBC+VT, wykorzystujący sterownik JDBC, lecz przy wykorzystaniu wątków wirtualnych, oferowanych przez Project Loom.

Badanie przeprowadzono dla każdego z powyższych wariantów. Wyniki eksperymentu przeprowadzonego w ramach scenariusza $\mathcal{A}$ przedstawiono w rozdziale szóstym, w sekcji 6.2.1 Wyniki pomiarów w ramach scenariusza $\mathcal{A}$.

Konstrukcja bazy danych

W każdym wariancie scenariusza testowego $\mathcal{A}$ wykorzystano relacyjną bazę danych PostgreSQL ze względu na wbudowane wsparcie dla równoległego przetwarzania zapytań, a także wsparcie połączenia za pośrednictwem sterowników JDBC i R2DBC [52]. Bazę danych uruchomiono w odrębnym środowisku przy wykorzystaniu konteneryzacji oferowanej przez narzędzie Docker [53]. Skonstruowano prosty schemat bazy danych oparty o system składania zamówień, przedstawiony na rysunku 5.1. W celu zapewnienia spójności danych i powtarzalności eksperymentu zdecydowano się na stworzenie deterministycznego generatora, który dla danej

wartości ziarna (ang. *seed*) generuje powtarzalny zestaw danych w bazie. Aby zredukować wpływ bazy danych na wynik eksperymentu, ustalono stałą maksymalną wartość równoczesnych połączeń z bazą danych, równą 32.



Rysunek 5.1: Schemat bazy danych wykorzystanej w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

Założenia eksperymentu

Dla każdego wariantu skonstruowano zestaw trzech zapytań do bazy danych:

- zapytanie Q1, będące prostym pobraniem wartości po kluczu głównym, przedstawione w listingu 1,
- zapytanie Q2, będące prostą zmianą wartości po kluczu głównym, przedstawione w listingu 2,
- zapytanie Q3, będące złożonym pobraniem wartości, wykorzystującym agregacje i wielokrotne złączenia, przedstawione w listingu 3.

```
SELECT name, email
FROM users
WHERE id = <ID>
```

Listing 1: Zapytanie Q1 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

```
UPDATE orders
SET total = total + 10
WHERE id = <ID>
```

Listing 2: Zapytanie Q2 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

```
SELECT COUNT(DISTINCT o.id),
       SUM(oi.quantity),
       SUM(oi.quantity * p.price)
FROM users u JOIN orders o ON o.user_id = u.id
      JOIN order_items oi ON oi.order_id = o.id
      JOIN products p ON p.id = oi.product_id
WHERE u.id = <ID>
```

Listing 3: Zapytanie Q3 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

Każdy wariant wykorzystuje generator liczb losowych ze stałą, ustaloną wartością ziarna do generowania kolejnych identyfikatorów wymaganych w zapytaniach (patrz miejsca oznaczone jako <ID> w listingach 1, 2 i 3), co gwarantuje deterministyczne odtworzenie sekwencji tych samych zapytań dla każdego wywołania testu o tej samej wartości ziarna.

Na potrzeby eksperymentu definiuje się pojęcie *poziomu współbieżności*. W zależności od wariantu eksperymentu fraza ta oznacza:

- dla wariantu JDBC+PT: rozmiar puli platformowych wątków,
- dla wariantu R2DBC: liczbę równocześnie aktywnych subskrypcji,
- dla wariantu JDBC+VT: liczbę tworzonych wirtualnych wątków.

Z racji, że scenariusz jest badany pod kątem różnych poziomów współbieżności, w każdym wariancie wszystkie wątki rozpoczynają wykonywanie zapytania w tym samym momencie, aby nie zaburzyć wyniku z powodu powolnego rozłożenia startu, a także zadbane o poprawne zamknięcie połączenia z bazą po wykonaniu każdego zapytania.

Opis implementacji wariantów scenariusza

Listing 4 prezentuje uproszczoną implementację współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+PT. Wariant ten wykorzystuje pulę platformowych wątków (patrz linie 3. i 8.–9.) oraz sterownik JDBC do wykonania zapytania do bazy danych (patrz linia 17.). Szczegółową implementację metody `executeQueryUsingJDBC`, wykonującej zapytanie przy użyciu sterownika JDBC,

przedstawiono w dalszej części, w listingu 8. Obiekty klasy `CountDownLatch`: `countDownLatchStart` i `countDownLatchDone` zapewniają wcześniej wspomniany wymóg rozpoczęcia wykonania zapytania przez wszystkie wątki w tym samym momencie. Wartości zmiennych `concurrency` i `query` są dostosowywane odpowiednio do badanego poziomu współbieżności i wykonywanego zapytania.

```

1  var concurrency = CONCURRENCY_LEVEL;
2  var query = QUERY;
3  var platformThreadFactory = Thread.ofPlatform().factory();
4  var countDownLatchStart = new CountDownLatch(1);
5  var countDownLatchDone = new CountDownLatch(concurrency);
6
7  try (
8      var executor = Executors.newFixedThreadPool(
9          concurrency, platformThreadFactory)
10 ) {
11     for (int i = 0; i < concurrency; i++) {
12         executor.submit(() -> {
13             try {
14                 countDownLatchStart.await();
15                 // wykonanie zapytania do bazy danych
16                 executeQueryUsingJDBC(query);
17                 countDownLatchDone.countDown();
18             } catch (Exception e) {
19                 ...
20             } finally {
21                 countDownLatchDone.countDown();
22             }
23         });
24     }
25 }
26 countDownLatchStart.countDown();
27 countDownLatchDone.await();

```

Listing 4: Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+PT

(źródło: opracowanie własne)

Listing 5 przedstawia uproszczoną implementację współbieżnego wykonania zapytania w wariancie R2DBC. Wykorzystuje ona w tym celu operatory strumieniowe, takie jak `range`, `flatMap`, `then` i `block` wraz z odpowiednim wywołaniem metody wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC (patrz linia 6.). Szczegółową implementację tej metody przedstawiono w dalszej części w listingu 7.

```

1  var concurrency = CONCURRENCY_LEVEL;
2  var query = QUERY;
3
4  Flux.range(1, concurrency)
5      // wykonanie zapytania do bazy danych
6      .flatMap(_ -> executeQueryUsingR2DBC(query), concurrency)
7      .then()
8      .block();

```

Listing 5: Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie R2DBC

(źródło: opracowanie własne)

Listing 6 prezentuje uproszczoną implementację wykonania zapytania w wariancie JDBC+VT. Jediną różnicą względem wariantu JDBC+PT jest wykorzystanie innego mechanizmu wykonawczego zadań (ang. *executor*), angażującego wirtualne wątki (patrz linia 7.) zgodnie z paradygmatem *thread-per-request*, o którym wspomniano w podrozdziale 3.2 Wirtualne wątki (*Virtual Threads*).

```

1  var concurrency = CONCURRENCY_LEVEL;
2  var query = QUERY;
3  var countDownLatchStart = new CountDownLatch(1);
4  var countDownLatchDone = new CountDownLatch(concurrency);
5
6  try (var executor =
7      Executors.newVirtualThreadPerTaskExecutor()
8  ) {
9      for (int i = 0; i < concurrency; i++) {
10         executor.submit(() -> {
11             try {
12                 countDownLatchStart.await();
13                 // wykonanie zapytania do bazy danych
14                 executeQueryUsingJDBC(query);
15                 countDownLatchDone.countDown();
16             } catch (Exception e) {
17                 ...
18             } finally {
19                 countDownLatchDone.countDown();
20             }
21         });
22     }
23 }
24 countDownLatchStart.countDown();
25 countDownLatchDone.await();

```

Listing 6: Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+VT

(źródło: opracowanie własne)

Jak wspomniano wyżej, wariant R2DBC scenariusza $\mathcal{A}$ do wykonania zapytania do bazy danych wykorzystuje metodę `executeQueryUsingR2DBC`, zaś warianty JDBC+PT i JDBC+VT wykorzystują w tym celu metodę `executeQueryUsingJDBC`. Ich uproszczoną implementację prezentują odpowiednio listingi 7 i 8.

Listing 7 przedstawia implementację metody wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC. Cechuje ją użycie podejścia reaktywnego, w tym operatorów strumieniowych. Metoda `executeQueryUsingR2DBC` wykorzystuje obiekt klasy `Blackhole`, oferowane przez narzędzie JMH (Java Microbenchmark Harness) i służące do przetwarzania danych w taki sposób, aby kompilator nie mógł zoptymalizować wyniku (patrz linie 3. i 17.). Narzędzie JMH i obiekt klasy `Blackhole` zostały szczegółowo opisane w podrozdziale 6.1 Metody pomiaru i kryteria oceny.

```

1      var connectionFactory = getConnectionFactoryForR2DBC();
2      var id = generateNextIdForQuery();
3      var blackhole = getBenchmarkBlackhole();
4
5      void executeQueryUsingR2DBC(String query) {
6          return Mono.whenWhen(
7              connectionFactory.create(),
8              connection -> {
9                  Flux.from(connection.createStatement(query)
10                      .bind("<ID>", id)
11                      .execute()
12                  )
13                  .flatMap(result ->
14                      result.map((row, _) -> {
15                          var value =
16                              row.get(COLUMN_NAME, String.class);
17                          blackhole.consume(value);
18                          ...
19                          // metoda zwraca cokolwiek - wynik jest
20                          // wcześniej konsumowany przez Blackhole
21                          return new Object();
22                      })
23                  )
24                  .then()
25              },
26              Connection::close
27          );
28      }

```

Listing 7: Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC w ramach scenariusza $\mathcal{A}$

(źródło: opracowanie własne)

Listing 8 przedstawia implementację metody wykonującej zapytanie do bazy danych przy użyciu sterownika JDBC. Reprezentuje on klasyczne podejście do nawiązywania połączenia z bazą danych i wykonania zapytania. Metoda ta również wykorzystuje wspomniany wcześniej obiekt klasy `Blackhole` (patrz linie 2. i 13.).

```

1  var id = generateNextIdForQuery();
2  var blackhole = getBenchmarkBlackhole();
3
4  void executeQueryUsingJDBC(String query) {
5      try (
6          var connection = getConnectionByJDBC();
7          var statement = connection.prepareStatement(query)
8      ) {
9          statement.setLong("<ID>", id);
10         try (var resultSet = statement.executeQuery()) {
11             while (resultSet.next()) {
12                 var value = resultSet.getString(1);
13                 blackhole.consume(value);
14                 ...
15             }
16         }
17     } catch (SQLException e) { ... }
18 }

```

Listing 8: Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika JDBC w ramach scenariusza $\mathcal{A}$

(źródło: opracowanie własne)

5.2.2 Scenariusz $\mathcal{B}$: Agregacja wyników oparta na mechanizmie kworum

Scenariusz $\mathcal{B}$ ma na celu zbadanie wpływu doboru mechanizmu oferującego współbieżne przetwarzanie danych, ze szczególnym uwzględnieniem mechanizmu współbieżności strukturalnej, w kontekście zadania polegającego na agregacji wyników opartej na mechanizmie kworum (ang. *quorum mechanism*). Mechanizm ten polega na podejmowaniu decyzji lub uznaniu operacji za zakończoną dopiero po uzyskaniu określonej liczby odpowiedzi (tzw. *wartość kworum*), zamiast oczekiwania na wszystkie odpowiedzi.

W analizowanym scenariuszu rozpatrywany jest zbiór pięciu niezależnych operacji, reprezentujących wywołania zewnętrznych serwisów. Operacje te są inicjowane równolegle i wykonywane współbieżnie. Każda z nich może zakończyć się sukcesem, zwracając wynik, lub niepowodzeniem. Celem przetwarzania jest uzyskanie co najmniej trzech poprawnych wyników (tj. wartość kworum jest równa 3). Osiągnięcie

tego progu traktowane jest jako warunek sukcesu całego procesu agregacji. W momencie spełnienia tego warunku dalsze oczekiwanie na pozostałe wyniki zostaje przerwane, a wszystkie niezakończone operacje są anulowane. W przypadku, gdy liczba poprawnie zakończonych operacji jest mniejsza niż wartość kworum, proces uznawany jest za zakończony niepowodzeniem.

Opis symulowanych serwisów

Warianty scenariusza testowego $\mathcal{B}$ wykorzystują różne typy serwisów. Każdy serwis jest definiowany przez zestaw następujących parametrów:

- D_{base} – wartość bazowego opóźnienia odpowiedzi serwisu, podana w milisekundach i wyznaczana z zadanego przedziału (ang. *base delay*),
- J_{base} – wartość odchylenia od bazowego opóźnienia, podana w milisekundach i wyznaczana z zadanego przedziału (ang. *jitter*),
- P_{outlier} – prawdopodobieństwo wystąpienia bardzo wolnej odpowiedzi, gdzie $0 \leq P_{\text{outlier}} \leq 1$ (ang. *outlier probability*),
- L_{outlier} – wartość opóźnienia podczas wystąpienia bardzo wolnej odpowiedzi, podana w milisekundach (ang. *outlier latency*).

Na potrzeby eksperymentu wartości bazowego opóźnienia (D_{base}) i odchylenia od tej wartości (J_{base}) oraz ustalenie wystąpienia bardzo wolnej odpowiedzi (P_{outlier}) są wyznaczane losowo przy użyciu ustalonej wartości ziarna, co gwarantuje deterministyczne odtworzenie dokładnych wartości opóźnień symulowanych serwisów dla każdego wariantu badanego modelu współbieżności.

Tabela 5.1 przedstawia cztery typy serwisów wraz z ich parametrami, symulujące różne rodzaje opóźnień i czasu przetwarzania. Serwisy **FAST** i **MEDIUM** charakteryzują się odpowiednio krótkim i umiarkowanie krótkim czasem odpowiedzi przy małym prawdopodobieństwie wystąpienia bardzo wolnej odpowiedzi. Serwis **SLOW** cechuje długi czas odpowiedzi przy nieco większym prawdopodobieństwie wystąpienia bardzo wolnej odpowiedzi niż poprzednie serwisy. Odmiennym zachowaniem charakteryzuje się serwis **OUTLIER_PRONE**, który bazowo zwraca odpowiedź w umiarkowanie krótkim czasie, lecz posiada największe prawdopodobieństwo wystąpienia bardzo wolnej odpowiedzi w porównaniu do pozostałych serwisów.

Typ serwisu	D_{base}	J_{base}	P_{outlier}	L_{outlier}
FAST	5–10 ms	2–4 ms	0.03	40 ms
MEDIUM	30–45 ms	5–10 ms	0.03	150 ms
SLOW	70–120 ms	10–20 ms	0.05	250 ms
OUTLIER_PRONE	25–40 ms	10–20 ms	0.2	300 ms

Tabela 5.1: Typy i parametry symulowanych serwisów wykorzystywanych w ramach scenariusza $\mathcal{B}$

(źródło: opracowanie własne)

W ramach przeprowadzonego eksperymentu skonstruowano i wykorzystano cztery zestawy serwisów, reprezentujące różne ich konfiguracje:

1. THREE_FAST_TWO_SLOW, wykorzystujący trzy serwisy typu FAST i dwa serwisy typu SLOW,
2. ONE_FAST_FOUR_SLOW, wykorzystujący jeden serwis typu FAST i cztery serwisy typu SLOW,
3. FIVE_SIMILAR, wykorzystujący pięć serwisów typu MEDIUM,
4. FIVE_OUTLIER_HEAVY, wykorzystujący pięć serwisów typu OUTLIER_PRONE.

Warianty modeli współbieżności i ich implementacja

W ramach eksperymentu zdecydowano się na zbadanie trzech wariantów umożliwiających implementację logiki zawartej w scenariuszu $\mathcal{B}$:

- Legacy, wykorzystujący w tym celu interfejs `ExecutorService` i natywne narzędzia upraszczające współbieżne przetwarzanie danych, w tym klasę `CompletableFuture`,
- Reactive, wykorzystujący w tym mechanizmy programowania asynchronicznego, dostępne w ramach narzędzia Reactive Streams,
- Loom, wykorzystujący w tym celu mechanizm współbieżności strukturalnej w oparciu o własną strategię synchronizacji podzadań.

Badanie przeprowadzono dla każdego z powyższych wariantów na każdym z czterech zestawów serwisów, opisanych w poprzednim akapicie. Wyniki eksperymentu przeprowadzonego w ramach scenariusza $\mathcal{B}$ przedstawiono w rozdziale szóstym, w podrozdziale 6.2.2 Wyniki pomiarów w ramach scenariusza $\mathcal{B}$.

Listingi 9 i 10 przedstawiają uproszczoną implementację logiki zawartej w scenariuszu $\mathcal{B}$ w wariantcie *Legacy*. Listing 9 prezentuje wysokopoziomowy mechanizm współbieżnego wywołania pięciu serwisów, a listing 10 zawiera właściwą logikę odpowiedzialną za zwrócenie wyniku lub wywołanie błędu agregacji. Wariant *Legacy* implementuje wariant agregacji wyników oparty na klasycznym mechanizmie wykorzystującym klasę `CompletableFuture` [54] i mechanizm wykonawczy zadań `ExecutorService` [16]. Dla każdej konfiguracji eksperymentu uruchamiane są równoległe wywołania wszystkich usług. Uzyskane odpowiedzi są odczytywane do momentu osiągnięcia wymaganego kworum. Każde wywołanie serwisu jest przekazywane do mechanizmu wykonawczego przez metodę `supplyAsync` (patrz linia 6. w listingu 9), natomiast oczekiwanie na odpowiedzi odbywa się poprzez blokujące wywołanie metody `get` (patrz linia 8. w listingu 10). Po osiągnięciu wartości kworum wyniki są agregowane, a pozostałe niezakończone zadania są anulowane. Warto zwrócić uwagę na fakt, że w wariantcie *Legacy* implementacja wymaga ręcznego anulowania wywołań serwisów w momencie zakończenia pracy poprzez blok `finally` (patrz linia 13. w listingu 9).

```

1      var quorum = 3;
2      var poolSize = 5;
3      var executor = Executors.newFixedThreadPool(poolSize);
4      var services = getServices(EXAMPLE_PRESET);
5      var futures = services.stream()
6          .map(service -> CompletableFuture.supplyAsync(
7              service::call, executor))
8          .toList();
9
10     try {
11         return aggregateQuorum(futures, quorum);
12     } finally {
13         futures.forEach(future -> future.cancel(true));
14     }

```

Listing 9: Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariantcie *Legacy*

(źródło: opracowanie własne)

```

1  String aggregateQuorum(
2      List<CompletableFuture<String>> futures,
3      int quorum
4  ) {
5      var successes = new ArrayList<String>(quorum);
6      for (var future : futures) {
7          try {
8              var result = future.get();
9              successes.add(result);
10             if (successes.size() >= quorum) {
11                 return aggregate(successes);
12             }
13         } catch (Exception e) { ... }
14     }
15
16     if (successes.size() >= quorum) {
17         return aggregate(successes);
18     }
19     throw new IllegalStateException("Quorum not achieved");
20 }

```

Listing 10: Metoda zawierająca właściwą logikę agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Legacy

(źródło: opracowanie własne)

Listing 11 prezentuje uproszczoną implementację logiki w ramach wariantu **Reactive**. Reprezentuje ona podejście reaktywne, w którym współbieżność i przetwarzanie odpowiedzi są realizowane przez operatory strumieniowe zamiast przez bezpośrednie zarządzanie obiektami typu `Future`, jak w przypadku wariantu **Legacy**. Dla każdej konfiguracji eksperymentu tworzony jest strumień danych za pomocą metody `Flux.fromIterable` (patrz linia 4.), w którym wywołania poszczególnych usług są uruchamiane równolegle w ramach metody `Mono.fromCallable` (patrz linia 6.). Operator `take` kończy zbieranie wyników po uzyskaniu wymaganej liczby poprawnych odpowiedzi, a błędne odpowiedzi są pomijane przez operator `onErrorResume`. W przypadku nieosiągnięcia wartości kworum zwracany jest obiekt `Mono.error`, informujący o błędzie agregacji (patrz linie 14.–20.). W przeciwnym wypadku zebrane wyniki są agregowane do pojedynczego wyniku tekstowego i zwracane synchronicznie za pomocą metody `block`.


```

1  var quorum = 3;
2  var services = getServices(EXAMPLE_PRESET);
3
4  return Flux.fromIterable(services)
5              .flatMap(service ->
6                  Mono.fromCallable(service::call)
7                      .subscribeOn(Schedulers.boundedElastic())
8                      .onErrorResume(_ -> Mono.empty()),
9                  services.size()
10             )
11     .take(quorum)
12     .collectList()
13     .flatMap(results -> {
14         if (results.size() < quorum) {
15             return Mono.error(
16                 new IllegalStateException(
17                     "Quorum not achieved"
18                 )
19             );
20         }
21         return Mono.just(aggregate(results));
22     })
23     .block();

```

Listing 11: Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza *B* w wariancie **Reactive**

(źródło: opracowanie własne)

Listingi 12 i 13 przedstawiają uproszczoną implementację logiki w ramach wariantu **Loom**. Wykorzystuje on mechanizm współbieżności strukturalnej i własną strategię agregacji wyników, uwzględnioną w ramach klasy `QuorumJoiner` (patrz listing 13). Dla każdej konfiguracji eksperymentu tworzony jest zakres zadań, w którym każda usługa wywoływana jest jako osobne podzadanie przy użyciu metody `fork` (patrz linia 7. w listingu 12). Do sterowania zakończeniem oczekiwania wykorzystywany jest niestandardowa strategia, zaimplementowana w ramach klasy `QuorumJoiner`, która zbiera poprawne odpowiedzi i kończy oczekiwanie po osiągnięciu wymaganej wartości kworum. Po zebraniu wyników za pomocą metody `join` następuje zwrócenie błędu agregacji w przypadku niez uzyskania wymaganej wartości kworum, a w przeciwnym wypadku agreguje i zwraca wynik.

```

1  var quorum = 3;
2  var services = getServices(EXAMPLE_PRESET);
3  var joiner = new QuorumJoiner(quorum);
4
5  try (var scope = StructuredTaskScope.open(joiner)) {
6      for (var service : services) {
7          scope.fork(service::call);
8      }
9      var successes = scope.join();
10     if (successes.size() < quorum) {
11         throw new IllegalStateException("Quorum not achieved");
12     }
13     return aggregate(successes);
14 }

```

Listing 12: Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom

(źródło: opracowanie własne)

```

1  public final class QuorumJoiner
2  implements StructuredTaskScope.Joiner<String, List<String>> {
3
4      private final int quorum;
5      private final ConcurrentLinkedQueue<String> successes;
6      private final AtomicInteger successCount;
7
8      public QuorumJoiner(int quorum) { ... }
9
10     @Override
11     public boolean onComplete(
12         Subtask<? extends String> subtask
13     ) {
14         if (subtask.state() == Subtask.State.SUCCESS) {
15             successes.add(subtask.get());
16         }
17         return successCount.incrementAndGet() >= quorum;
18     }
19
20     @Override
21     public List<String> result() {
22         return List.copyOf(successes);
23     }
24 }

```

Listing 13: Implementacja własnej strategii agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom

(źródło: opracowanie własne)

5.3 Hipotezy badawcze

Rozdział 6

Wyniki badań i ich analiza

Szósty rozdział pracy przedstawia kryteria oceny wyników badań przeprowadzonych w ramach scenariuszy $\mathcal{A}$ i $\mathcal{B}$, w tym definiuje przyjęte metryki, zawiera szczegółowy opis pomiarów, prezentuje uzyskane wyniki, a także przedstawia ich analizę porównawczą i interpretację.

6.1 Kryteria oceny wyników

Badania zdefiniowane w ramach scenariuszy $\mathcal{A}$ i $\mathcal{B}$ przeprowadzono w postaci benchmarków. W tym celu wykorzystano narzędzie Java Microbenchmark Harness (JMH), stanowiące standardowy framework służący do przeprowadzania pomiarów wydajności aplikacji działających w środowisku JVM. Narzędzie to umożliwia wykonywanie powtarzalnych i możliwie precyzyjnych pomiarów czasu wykonania kodu, uwzględniając przy tym specyfikę działania maszyny wirtualnej Java, w szczególności mechanizmy takie jak kompilacja Just-In-Time (JIT), optymalizacje wykonywanego kodu czy proces rozgrzewania środowiska uruchomieniowego [55].

Użyto również dwóch narzędzi do statycznej analizy kodu: SonarQube i PMD, [opis narzędzi jak wyżej].

6.1.1 Opis użytych trybów benchmarków i definicja mierzonych przez nie metryk

Do uruchomienia benchmarków przeprowadzonych na podstawie scenariuszy $\mathcal{A}$ i $\mathcal{B}$ wykorzystano dwa tryby: `Throughput` i `SampleTime`.

Tryb pomiaru przepustowości (`Throughput`)

Tryb `Throughput` w JMH służy do pomiaru przepustowości badanego fragmentu kodu, określając liczbę operacji wykonanych w jednostce czasu (w przypadku

scenariusza $\mathcal{A}$ jednostką czasu jest sekunda). Tryb ten koncentruje się na całkowitej zdolności systemu do przetwarzania zadań przy zadanym obciążeniu, co w badanym scenariuszu pozwoliło na analizę wydajności i skalowalności różnych wariantów systemów współbieżnych. [56]

Z powyższego opisu wynika, że metryką mierzoną przez benchmark uruchomiony w trybie `Throughput` jest *przepustowość*, czyli liczba operacji wykonanych przez badany fragment kodu w jednostce czasu, oznaczana jako THRPT. Tryb ten został użyty przy pomiarze kodu zaimplementowanego w ramach scenariusza $\mathcal{A}$.

Tryb próbkowania czasu (`SampleTime`)

Tryb `SampleTime` w JMH służy do pomiaru czasu wykonania pojedynczych wywołań benchmarkowanej metody na podstawie próbek zbieranych w trakcie iteracji pomiarowych. Wynikiem jest średni czas wykonania operacji wraz z oszacowaniem błędu pomiarowego. Dodatkowo JMH raportuje rozkład zebranych próbek, w tym wartości minimalne i maksymalne oraz percentyle. Wartości te pozwalają ocenić latencję i zachowanie w ogonie rozkładu [56]. Tryb ten został użyty przy pomiarze kodu zaimplementowanego w ramach obu scenariuszy.

Zgodnie z definicją sformułowaną przez Luszniewicza [57], mianem *percentyla rzędu p* , gdzie $p \in [0, 100]$, określa się taką wartość x , że co najmniej $p\%$ obserwacji ma wartość mniejszą lub równą x , a co najwyżej $(100 - p)\%$ obserwacji ma wartość większą lub równą x .

Na podstawie powyższej definicji, metryką P_n określa się percentyl rzędu n czasu wykonania badanego fragmentu kodu. Ponadto, metryką AVG określa się średni czas wykonania badanego fragmentu kodu.

6.1.2 Definicja metryk statycznej analizy kodu

6.2 Szczegółowy opis pomiarów

6.2.1 Szczegóły pomiaru dla scenariusza $\mathcal{A}$

Każdy z trzech wariantów scenariusza $\mathcal{A}$ (JDBC+PT, R2DBC, JDBC+VT) został uruchomiony dla każdego z trzech zapytań (Q1, Q2, Q3) i na różnych poziomach współbieżności (16, 32, 64, 256, 1024) w następującej konfiguracji narzędzia JMH:

- `BenchmarkMode` – tryby benchmarku: `Throughput` i `SampleTime`,
- `Warmup` – liczba iteracji rozgrzewkowych¹: 5, każda trwająca 10 sekund,

¹Iteracje rozgrzewkowe pozwalają na ustabilizowanie działania środowiska uruchomieniowego JVM, m.in. poprzez aktywację kompilatora JIT i optymalizacji wykonywanego kodu, przed rozpoczęciem właściwych pomiarów. [56]

- **Measurement** – liczba iteracji pomiarów właściwych: 10, każda trwająca 10 sekund,
- **Fork** – liczba rozgałęzień²: 3,
- **Threads** – liczba wątków jednocześnie wykonujących kod: 1.

Liczba wątków w powyższej konfiguracji została ustawiona na 1 ze względu na fakt, że zarządzanie liczbą i rodzajem wątków zostało przeniesione bezpośrednio do metod, na których wykonywane są benchmarki. Parametr ten reprezentowany jest przez poziom współbieżności. Implementacja tej logiki różni się w zależności od wariantu modelu współbieżności, co zostało szczegółowo przedstawione w sekcji 5.2.1 Scenariusz $\mathcal{A}$: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC).

W celu uniknięcia dokonania niezamierzonych optymalizacji przez środowisko JVM, każdy wariant wykorzystuje wspomniane w rozdziale piątym narzędzie **Blackhole** dostępne w ramach JMH, konsumujące wyniki obliczeń w sposób, którego kompilator nie może bezpiecznie zoptymalizować. W ten sposób narzędzie to pozwala zapewnić, że mierzony kod faktycznie zostaje wykonany, a uzyskane wyniki benchmarku odzwierciedlają rzeczywisty koszt obliczeń. [58]

Metrykami mierzonymi w ramach scenariusza $\mathcal{A}$ są:

- dla trybu **Throughput**: THRPT,
- dla trybu **SampleTime**: AVG, P50, P95 i P99,
- w ramach statycznej analizy kodu: ?.

6.2.2 Szczegóły pomiaru dla scenariusza $\mathcal{B}$

Metrykami mierzonymi w ramach scenariusza $\mathcal{B}$ są:

- dla trybu **SampleTime**: AVG, P50, P95, P99 i P99.9,
- w ramach statycznej analizy kodu: ?.

6.3 Prezentacja wyników pomiarów

6.3.1 Wyniki pomiarów w ramach scenariusza $\mathcal{A}$

Tryb pomiaru przepustowości (Throughput)

Tabele 6.1, 6.2 i 6.3 oraz odpowiadające im wykresy przedstawione na rysunkach 6.1, 6.2 i 6.3 prezentują wyniki benchmarku przeprowadzonego dla scenariusza $\mathcal{A}$

²Rozgałęzienia to liczba niezależnych uruchomień środowiska JVM, w których wykonywany jest pomiar. [56]

w trybie pomiaru przepustowości, tj. w trybie **Throughput**. Wartości przedstawione w tabelach i na odpowiadających im wykresach zostały wyznaczone w zależności od wariantu modelu współbieżności, poziomu współbieżności i zapytania.

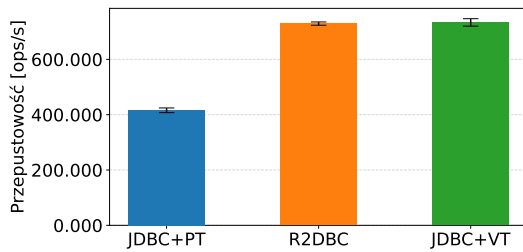
Dodatkowe uwagi do tabel 6.1, 6.2 i 6.3:

1. Wyniki we wspomnianych wyżej tabelach podane są jako liczba operacji na sekundę. Wykresy przedstawione na odpowiadających im rysunkach prezentują te dane na osi pionowej w jednostce oznaczonej jako [ops/s].
2. Wyniki we wspomnianych wyżej tabelach przedstawione są w formacie $S \pm E$, gdzie S oznacza uzyskany wynik, a E oznacza błąd pomiaru.
3. Wyniki i wartości błędów pomiaru we wspomnianych wyżej tabelach zapisane zostały z kropką jako separatorem dziesiętnym i zmierzone z dokładnością do części tysięcznych, tj. 0.001.

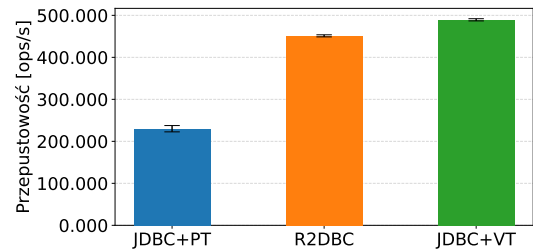
Poz. współb.	JDBC+PT	R2DBC	JDBC+VT
16	415.881 ± 8.497	729.356 ± 5.935	733.642 ± 13.414
32	230.098 ± 7.511	451.219 ± 2.474	489.411 ± 2.623
64	116.672 ± 4.463	240.370 ± 4.193	250.862 ± 0.876
256	28.871 ± 1.067	67.178 ± 0.352	66.336 ± 0.378
1024	6.694 ± 0.306	17.139 ± 0.105	16.782 ± 0.111

Tabela 6.1: Wartości przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

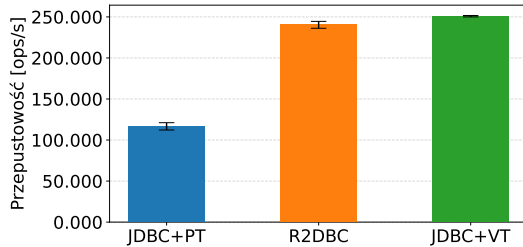
(źródło: opracowanie własne)



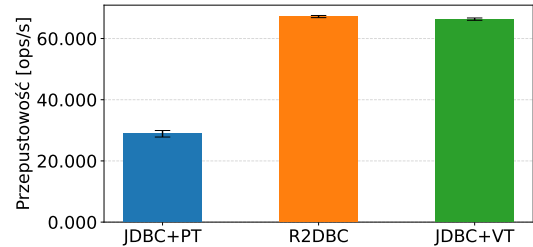
(a) poziom współbieżności: 16



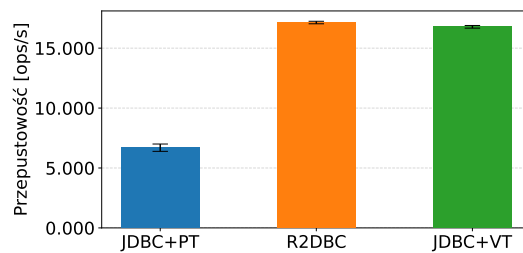
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

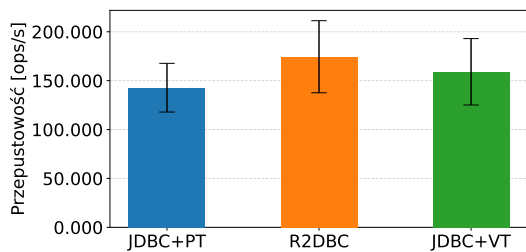
Rysunek 6.1: Wykresy kolumnowe przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

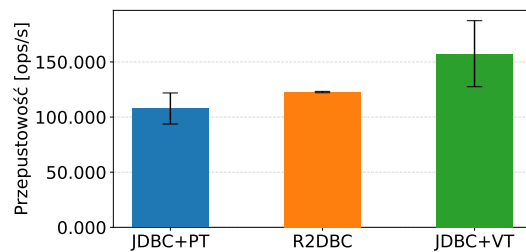
Poz. współb.	JDBC+PT	R2DBC	JDBC+VT
16	142.817 $\pm$ 24.898	174.474 $\pm$ 36.850	159.069 $\pm$ 33.964
32	107.766 $\pm$ 14.097	122.683 $\pm$ 0.635	157.456 $\pm$ 29.941
64	55.572 $\pm$ 5.507	74.120 $\pm$ 10.150	70.235 $\pm$ 4.447
256	14.765 $\pm$ 1.268	19.392 $\pm$ 2.232	19.884 $\pm$ 1.825
1024	3.736 $\pm$ 0.365	4.600 $\pm$ 0.031	5.042 $\pm$ 0.379

Tabela 6.2: Wartości przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

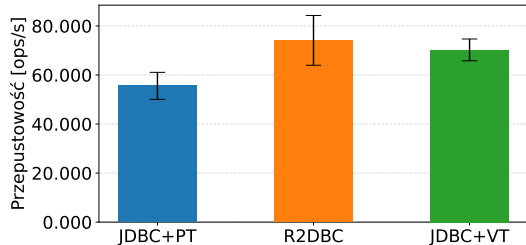
(źródło: opracowanie własne)



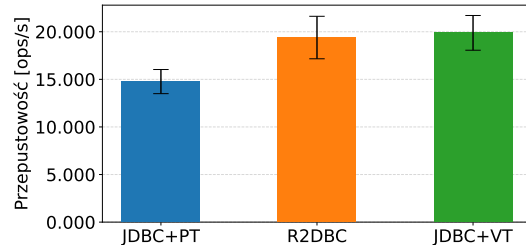
(a) poziom współbieżności: 16



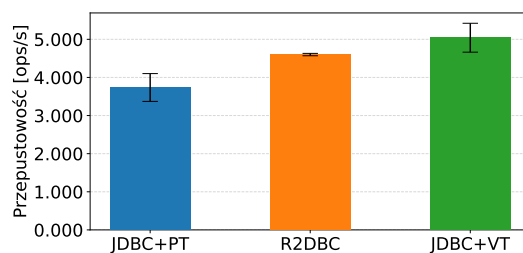
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

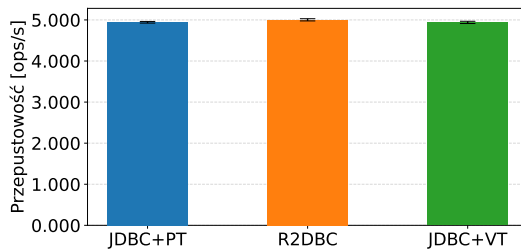
Rysunek 6.2: Wykresy kolumnowe przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

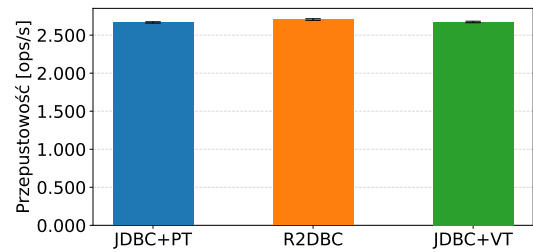
Poz. współł.	JDBC+PT	R2DBC	JDBC+VT
16	4.943 ± 0.020	5.003 ± 0.027	4.940 ± 0.027
32	2.667 ± 0.009	2.705 ± 0.011	2.673 ± 0.010
64	1.319 ± 0.004	1.332 ± 0.006	1.314 ± 0.008
256	0.330 ± 0.002	0.335 ± 0.001	0.330 ± 0.002
1024	0.187 ± 0.001	0.084 ± 0.001	0.192 ± 0.001

Tabela 6.3: Wartości przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

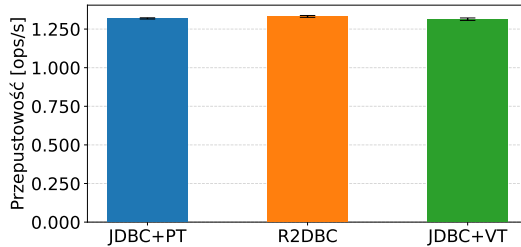
(źródło: opracowanie własne)



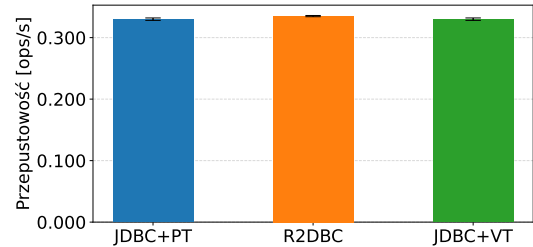
(a) poziom współbieżności: 16



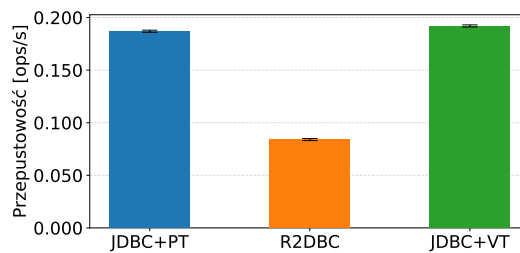
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

Rysunek 6.3: Wykresy kolumnowe przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Tryb próbkowania czasu (SampleTime)

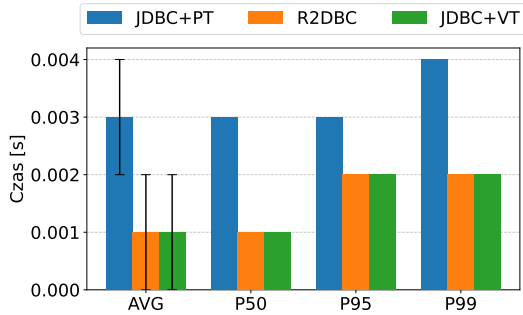
Dodatkowe uwagi do tabel 6.4, 6.5 i 6.6:

1. Wyniki we wspomnianych wyżej tabelach uwzględniają średni czas (AVG) i wartości percentyli P50, P95, P99 czasu wykonania badanych wariantów scenariusza i podane są w sekundach. Wykresy przedstawione na na odpowiadających im rysunkach prezentują te dane na osi pionowej w jednostce oznaczonej jako [s].
2. Wyniki we wierszach oznaczonych przez AVG we wspomnianych wyżej tabelach są przedstawione w formacie $S \pm E$, gdzie S oznacza uzyskany wynik, a E oznacza błąd pomiaru.
3. Wyniki i wartości błędów we wspomnianych wyżej tabelach zapisane zostały z kropką jako separatorem dziesiętnym, z dokładnością do części tysięcznych, tj. 0.001.

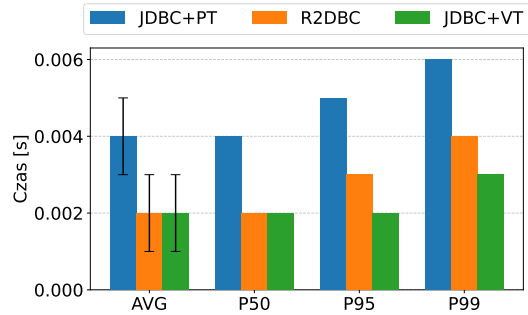
Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.003 ± 0.001	0.001 ± 0.001	0.001 ± 0.001
	P50	0.003	0.001	0.001
	P95	0.003	0.002	0.002
	P99	0.004	0.002	0.002
32	AVG	0.004 ± 0.001	0.002 ± 0.001	0.002 ± 0.001
	P50	0.004	0.002	0.002
	P95	0.005	0.003	0.002
	P99	0.006	0.004	0.003
64	AVG	0.009 ± 0.001	0.004 ± 0.001	0.004 ± 0.001
	P50	0.009	0.004	0.004
	P95	0.010	0.005	0.005
	P99	0.011	0.006	0.006
256	AVG	0.036 ± 0.001	0.015 ± 0.001	0.015 ± 0.001
	P50	0.035	0.015	0.015
	P95	0.039	0.017	0.018
	P99	0.043	0.020	0.020
1024	AVG	0.159 ± 0.001	0.058 ± 0.001	0.060 ± 0.001
	P50	0.158	0.057	0.058
	P95	0.177	0.065	0.066
	P99	0.187	0.070	0.071

Tabela 6.4: Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

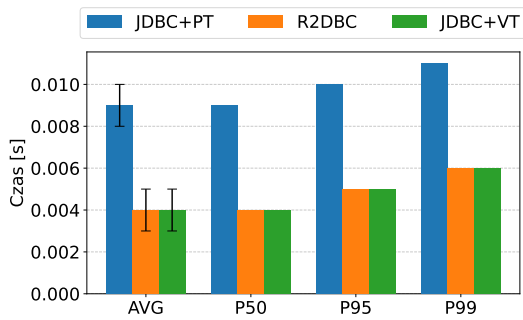
(źródło: opracowanie własne)



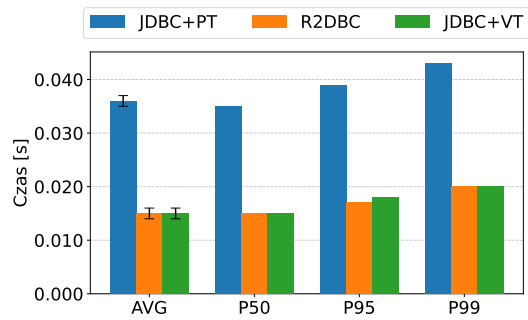
(a) poziom współbieżności: 16



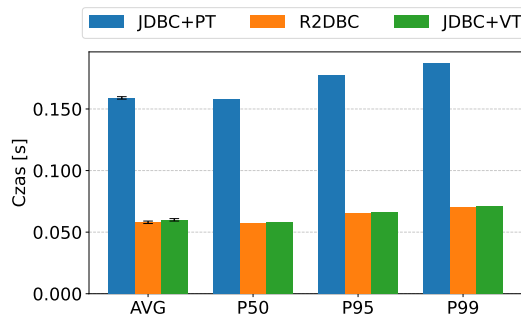
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

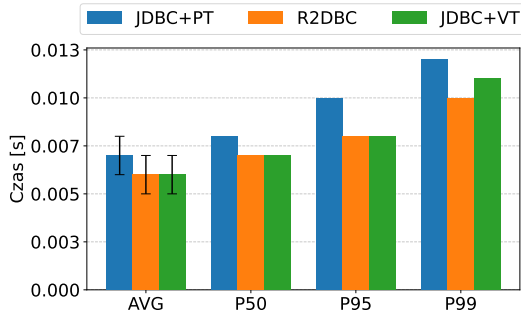
Rysunek 6.4: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

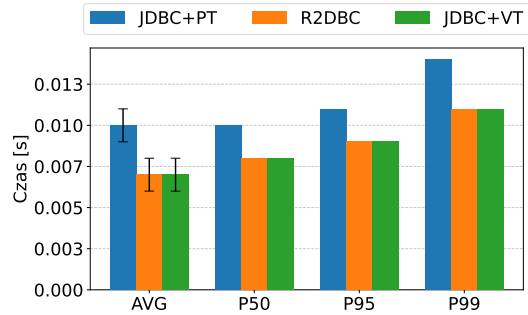
Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.007 ± 0.001	0.006 ± 0.001	0.006 ± 0.001
	P50	0.008	0.007	0.007
	P95	0.010	0.008	0.008
	P99	0.012	0.010	0.011
32	AVG	0.010 ± 0.001	0.007 ± 0.001	0.007 ± 0.001
	P50	0.010	0.008	0.008
	P95	0.011	0.009	0.009
	P99	0.014	0.011	0.011
64	AVG	0.019 ± 0.001	0.013 ± 0.001	0.014 ± 0.001
	P50	0.019	0.014	0.014
	P95	0.021	0.016	0.016
	P99	0.025	0.020	0.022
256	AVG	0.068 ± 0.001	0.054 ± 0.001	0.053 ± 0.001
	P50	0.071	0.054	0.052
	P95	0.077	0.059	0.056
	P99	0.096	0.081	0.079
1024	AVG	0.297 ± 0.003	0.214 ± 0.001	0.200 ± 0.002
	P50	0.299	0.212	0.202
	P95	0.329	0.236	0.223
	P99	0.364	0.264	0.257

Tabela 6.5: Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

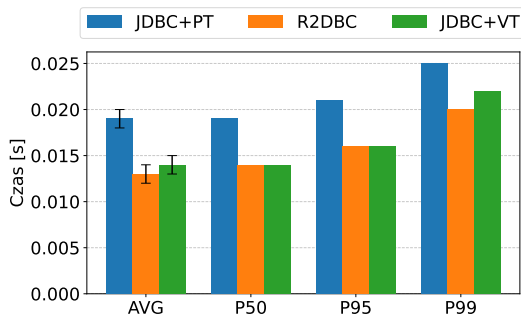
(źródło: opracowanie własne)



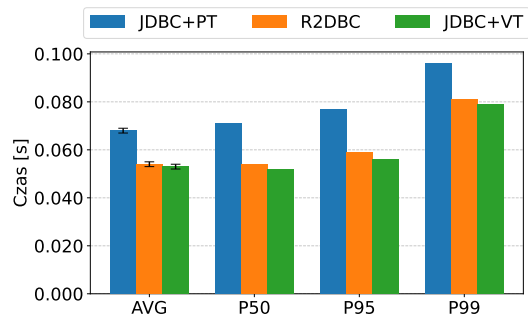
(a) poziom współbieżności: 16



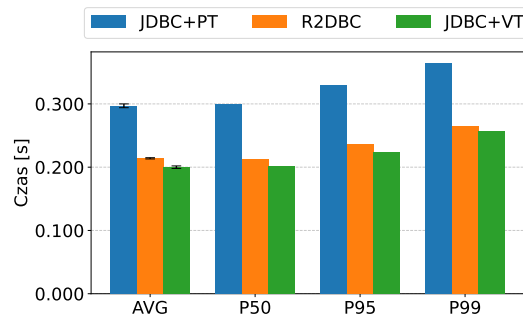
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

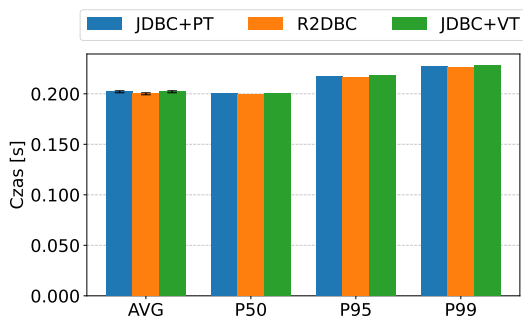
Rysunek 6.5: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

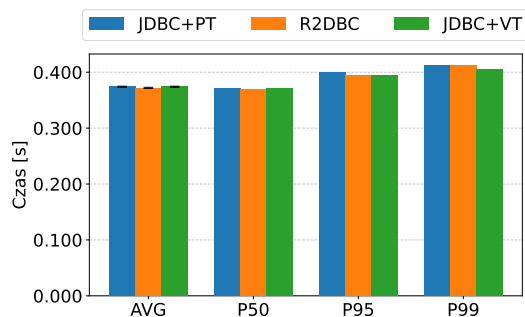
Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.202 ± 0.001	0.200 ± 0.001	0.202 ± 0.001
	P50	0.200	0.199	0.200
	P95	0.217	0.216	0.218
	P99	0.227	0.226	0.228
32	AVG	0.374 ± 0.001	0.372 ± 0.001	0.374 ± 0.001
	P50	0.371	0.369	0.372
	P95	0.400	0.394	0.394
	P99	0.412	0.412	0.406
64	AVG	0.757 ± 0.003	0.751 ± 0.003	0.760 ± 0.003
	P50	0.753	0.748	0.755
	P95	0.784	0.778	0.793
	P99	0.820	0.820	0.830
256	AVG	3.023 ± 0.012	2.999 ± 0.016	3.007 ± 0.013
	P50	3.012	2.991	3.001
	P95	3.091	3.078	3.095
	P99	3.176	3.311	3.148
1024	AVG	5.336 ± 0.010	11.954 ± 0.054	5.206 ± 0.009
	P50	5.335	11.929	5.201
	P95	5.377	12.148	5.234
	P99	5.385	12.231	5.260

Tabela 6.6: Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

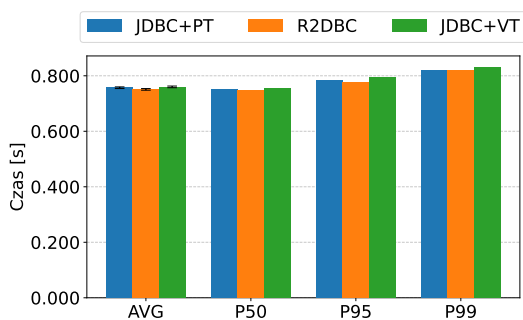
(źródło: opracowanie własne)



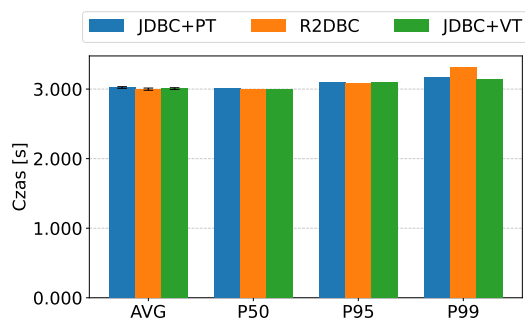
(a) poziom współbieżności: 16



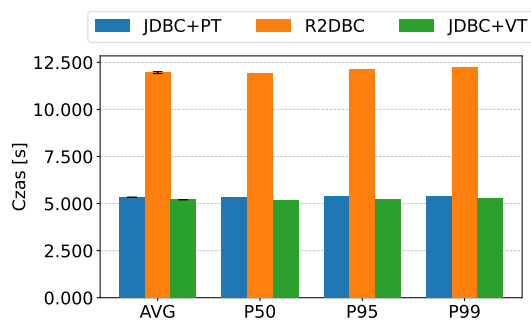
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

Rysunek 6.6: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Statyczna analiza kodu

bla bla bla

6.3.2 Wyniki pomiarów w ramach scenariusza $\mathcal{B}$

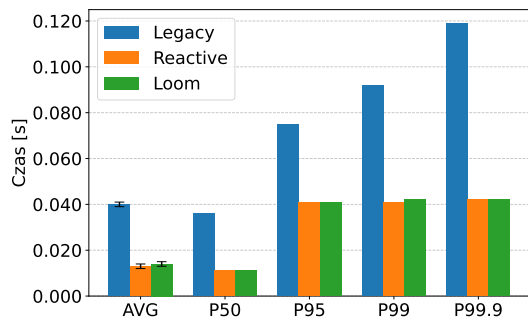
Tryb próbkowania czasu (SampleTime)

bla bla bla

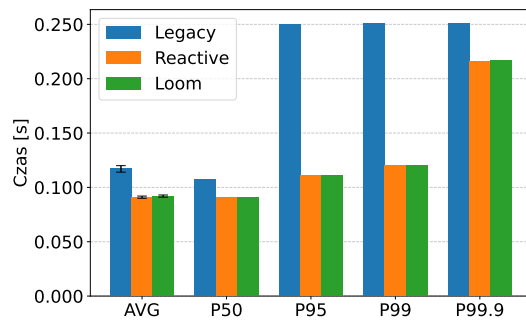
Wariant serwisów	Metryka	Legacy	Reactive	Loom
THREE_FAST_TWO_SLOW	AVG	0.040 ± 0.001	0.013 ± 0.001	0.014 ± 0.001
	P50	0.036	0.011	0.011
	P95	0.075	0.041	0.041
	P99	0.092	0.041	0.042
	P99.9	0.119	0.042	0.042
ONE_FAST_FOUR_SLOW	AVG	0.117 ± 0.003	0.091 ± 0.001	0.092 ± 0.001
	P50	0.107	0.091	0.091
	P95	0.250	0.111	0.111
	P99	0.251	0.120	0.120
	P99.9	0.252	0.216	0.217
FIVE_SIMILAR	AVG	0.053 ± 0.001	0.039 ± 0.001	0.039 ± 0.001
	P50	0.045	0.038	0.039
	P95	0.150	0.045	0.045
	P99	0.151	0.048	0.048
	P99.9	0.153	0.051	0.052
OUTLIER_HEAVY	AVG	0.166 ± 0.010	0.050 ± 0.003	0.051 ± 0.003
	P50	0.058	0.037	0.037
	P95	0.301	0.300	0.300
	P99	0.324	0.301	0.301
	P99.9	0.341	0.302	0.302

Tabela 6.7: Wartości średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od wariantu serwisów i modelu współbieżności

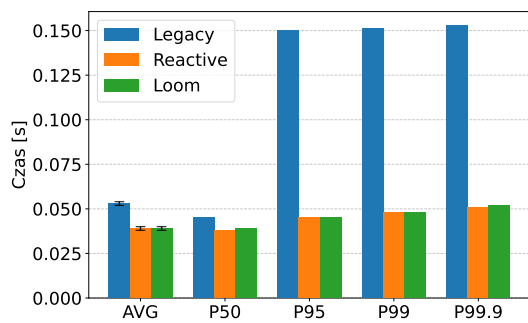
(źródło: opracowanie własne)



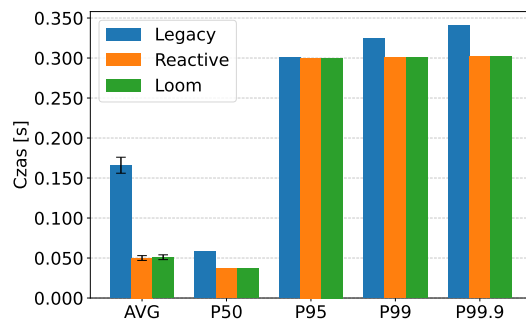
(a) wariant THREE_FAST_TWO_SLOW



(b) wariant ONE_FAST_FOUR_SLOW



(c) wariant FIVE_SIMILAR



(d) wariant OUTLIER_HEAVY

Rysunek 6.7: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od wariantu serwisów i modelu współbieżności

(źródło: opracowanie własne)

Statyczna analiza kodu

6.4 Analiza porównawcza i interpretacja wyników

6.4.1 Scenariusz $\mathcal{A}$

6.4.2 Scenariusz $\mathcal{B}$

Rozdział 7

Wnioski i rekomendacje

- 7.1 Odpowiedzi na pytania badawcze i weryfikacja hipotez badawczych
- 7.2 Rekomendacje i wzorce zastosowań mechanizmów Project Loom
- 7.3 Ograniczenia badań i propozycje dalszych badań

Rozdział 8

Podsumowanie

Bibliografia

- [1] Andrew S. Tanenbaum i Herbert Bos. *Modern Operating Systems*. Wydanie V. Pearson Education, 2023, s. 85–178.
- [2] Mordechai Ben-Ari. *Principles of Concurrent Programming*. Prentice-Hall International, 1982, s. 1–15, 18–27, 29–43, 51–55, 109–116.
- [3] Maurice Herlihy i Nir Shavit. *The Art of Multiprocessor Programming*. Elsevier, 2012, s. 6–10, 21–44.
- [4] Vincent Gramoli. *More than you ever wanted to know about synchronization: Synchrobench, measuring the impact of the synchronization on concurrent algorithms*. URL: <http://sydney.edu.au/engineering/it/~gramoli/doc/pubs/gramoli-synchrobench.pdf> (dostęp 27.03.2026).
- [5] Brian Goetz i in. *Java Concurrency in Practice*. Pearson Education, 2006, s. 1–9, 15–32, 113–115, 167–180, 205–220, 221–241.
- [6] Abraham Silberschatz, Peter B. Galvin i Greg Gagne. *Podstawy systemów operacyjnych, Tom I*. Wydanie X (I w WN PWN). Wydawnictwo Naukowe PWN, 2021.
- [7] Microsoft Learn. *Warunki wyścigu i zakleszczenia*. Przetłumaczone oryginalnie z języka angielskiego. URL: <https://learn.microsoft.com/pl-pl/troubleshoot/developer/visualstudio/visual-basic/language-compilers/race-conditions-deadlocks> (dostęp 28.03.2026).
- [8] Edsger W. Dijkstra. *Twenty-eight years*. Transkrypcja odręcznych notatek z 1965 roku. URL: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD10xx/EWD1000.html> (dostęp 28.03.2026).
- [9] Jeffrey C. Mogul i Anita Borg. *The Effect of Context Switches on Cache Performance*. ACM SIGPLAN Notices, 1991, s. 75–84. URL: <https://dl.acm.org/doi/pdf/10.1145/106973.106982>.
- [10] Dokumentacja klasy `java.lang.Thread` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html> (dostęp 5.04.2026).

- [11] Dokumentacja interfejsu `java.lang Runnable` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Runnable.html> (dostęp 5.04.2026).
- [12] Jeff Friesen. *Java Threads and the Concurrency Utilities*. Apress, 2015, s. 3–49.
- [13] Ron Veen i David Vlijmincx. *Virtual Threads, Structured Concurrency, and Scoped Values: Explore Java's New Threading Model*. Apress Berkeley, CA, 2024.
- [14] Doug Lea. *Concurrent Programming in Java: Design Principles and Patterns, Second Edition*. Addison Wesley, 1999, s. 5–31, 32–37.
- [15] Dokumentacja słowa kluczowego `synchronized` w specyfikacji języka Java. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html#jls-14.19> (dostęp 5.04.2026).
- [16] Dokumentacja interfejsu `java.util.concurrent.ExecutorService` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ExecutorService.html> (dostęp 5.04.2026).
- [17] Scott L. Bain. *The Design Patterns Companion*. Project Management Institute, 2020, s. 48–49, 50–51.
- [18] Dokumentacja klasy `java.util.concurrent.ThreadPoolExecutor` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html> (dostęp 6.04.2026).
- [19] Dokumentacja klasy `java.lang.Runtime` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Runtime.html> (dostęp 6.04.2026).
- [20] Strona internetowa inicjatywy Reactive Streams. URL: <https://www.reactive-streams.org> (dostęp 11.04.2026).
- [21] Strona internetowa projektu Project Reactor. URL: <https://projectreactor.io> (dostęp 11.04.2026).
- [22] Kod źródłowy projektu RxJava. URL: <https://github.com/ReactiveX/RxJava> (dostęp 11.04.2026).
- [23] Dokumentacja projektu Akka Streams. URL: <https://doc.akka.io/libraries/akka-core/current/stream/index.html> (dostęp 11.04.2026).
- [24] Oleh Dokuka i Igor Lozynskyi. *Hands-On Reactive Programming in Spring 5*. Packt Publishing, 2018, s. 6–40, 49–56, 72.

-
- [25] Dokumentacja klasy `java.util.concurrent.Flow` dla JDK w wersji 9. URL: <https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html> (dostęp 12.04.2026).
 - [26] Erich Gamma i in. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995, s. 293–304.
 - [27] Jonas Bonér i in. *The Reactive Manifesto*. Wersja druga. 2014. URL: <https://reactivemanifesto.org> (dostęp 11.04.2026).
 - [28] Adam L. Davis. *Reactive Streams in Java: Concurrency with RxJava, Reactor, and Akka Streams*. Apress, 2019.
 - [29] OpenJDK. *JEP 1: JDK Enhancement-Proposal & Roadmap Process*. URL: <https://openjdk.org/jeps/1> (dostęp 1.11.2025).
 - [30] OpenJDK. *JEP 444: Virtual Threads*. URL: <https://openjdk.org/jeps/444> (dostęp 1.11.2025).
 - [31] OpenJDK. *JEP 525: Structured Concurrency (Sixth Preview)*. URL: <https://openjdk.org/jeps/525> (dostęp 1.11.2025).
 - [32] OpenJDK. *JEP 506: Scoped Values*. URL: <https://openjdk.org/jeps/506> (dostęp 1.11.2025).
 - [33] Ron Pressler (OpenJDK). *Project Loom: Fibers and Continuations for the Java Virtual Machine*. URL: <https://cr.openjdk.org/~rpressler/loom/Loom-Proposal.html> (dostęp 12.12.2025).
 - [34] John D.C. Little i Stephen C. Graves. *Little's Law*. URL: <https://web.archive.org/web/20100215131801/http://web.mit.edu/sgraves/www/papers/Little's%20Law-Published.pdf> (dostęp 26.01.2015).
 - [35] Thomas H. Cormen i in. *Introduction to Algorithms, Third Edition*. Massachusetts Institute of Technology, 2009, s. 232–236.
 - [36] Alan Bateman (OpenJDK). *Project Loom: The challenges of introducing Virtual Threads to the Java Platform*. URL: <https://cr.openjdk.org/~alanb/jvmls2023/loom-jvmls2023.pdf> (dostęp 11.03.2026).
 - [37] Dokumentacja klasy `java.util.concurrent.Semaphore` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/Semaphore.html> (dostęp 6.04.2026).
 - [38] Dokumentacja klasy `java.util.concurrent.StructuredTaskScope` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/StructuredTaskScope.html> (dostęp 12.04.2026).

- [39] Oracle (Dokumentacja języka Java). *The try-with-resources Statement*. URL: <https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html> (dostęp 13.04.2026).
- [40] Dokumentacja interfejsu `java.util.concurrent.StructuredTaskScope.Joiner` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/StructuredTaskScope.Joiner.html> (dostęp 12.04.2026).
- [41] Dokumentacja klasy `java.lang.ThreadLocal` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ThreadLocal.html> (dostęp 13.04.2026).
- [42] Dokumentacja klasy `java.lang.ScopedValue` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ScopedValue.html> (dostęp 13.04.2026).
- [43] Sinthujan Ponnampalam. *Evaluation of Virtual Threads and RxJava: A performance and code complexity analysis in a real-time clearing system*. Umeå University. 2025.
- [44] Vishesh Pandita. *Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads*. National College of Ireland. 2024.
- [45] Elias Gustafsson i Oliver Nederlund Persson. *Comparison of Concurrency Technologies in Java*. Lund University. 2024.
- [46] Sebastian Iwanowski i Grzegorz Kozieł. *Analiza porównawcza podejścia reaktywnego i imperatywnego w tworzeniu aplikacji internetowych w języku Java*. Journal of Computer Sciences Institute. 2022.
- [47] Yazeed Al-Dossari i Layla Al-Fahad. *Java Concurrency Demystified: Thread Pools, Completable Future, and Virtual Threads*. International Journal of Informatics and Data Science Research. 2024.
- [48] Arooba Shahoor, Jooyong Yi i Dongsun Kim. *Preserving Reactiveness: Understanding and Improving the Debugging Practice of Blocking-call Bugs*. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024). 2024.
- [49] Andy Georges, Dries Buytaert i Lieven Eeckhout. *Statistically Rigorous Java Performance Evaluation*. ACM. 2007.
- [50] Dokumentacja API narzędzia JDBC. URL: <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc> (dostęp 10.04.2026).
- [51] Strona internetowa projektu R2DBC. URL: <https://r2dbc.io> (dostęp 10.04.2026).

-
- [52] Strona internetowa bazy danych PostgreSQL. URL: <https://www.postgresql.org/about> (dostęp 10.04.2026).
 - [53] Strona internetowa narzędzia Docker. *Use containers to Build, Share and Run your applications*. URL: <https://www.docker.com/resources/what-container> (dostęp 5.05.2026).
 - [54] Dokumentacja klasy `java.util.concurrent.CompletableFuture` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/CompletableFuture.html> (dostęp 7.05.2026).
 - [55] Kod źródłowy narzędzia Java Microbenchmark Harness (JMH). URL: <https://github.com/openjdk/jmh> (dostęp 9.04.2026).
 - [56] Dokumentacja API narzędzia Java Microbenchmark Harness (JMH) w wersji 1.37. URL: <https://javadoc.io/doc/org.openjdk.jmh/jmh-core/1.37/index.html> (dostęp 9.04.2026).
 - [57] Andrzej Luszczewicz. *Statystyka ogólna*. Państwowe Wydawnictwo Ekonomiczne, 1970, s. 9–15.
 - [58] Dokumentacja klasy `Blackhole` z narzędzia JMH w wersji 1.37. URL: <https://javadoc.io/doc/org.openjdk.jmh/jmh-core/latest/org/openjdk/jmh/infra/Blackhole.html> (dostęp 30.12.2025).

Spis rysunków

2.1	Rozróżnienie procesów i wątków w systemie operacyjnym	12
2.2	Ilustracja zjawiska <i>race condition</i> na podstawie wywołania procedury <code>increaseByOne</code>	14
2.3	Wywołanie procedury <code>increaseByOne</code> z wykorzystaniem synchronizacji wątków	14
2.4	Ilustracja problemu ucztujących filozofów	15
3.1	Cykl życia wirtualnych wątków w środowisku JVM	26
3.2	Schemat działania zakresu zadań w ramach klasy <code>StructuredTaskScope</code> w środowisku JVM	29
5.1	Schemat bazy danych wykorzystanej w ramach scenariusza $\mathcal{A}$	39
6.1	Wykresy kolumnowe przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	57
6.2	Wykresy kolumnowe przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	58
6.3	Wykresy kolumnowe przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	59
6.4	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	62
6.5	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	64
6.6	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	66

6.7	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od wariantu serwisów i modelu współbieżności	70
-----	---	----

Spis tabel

5.1	Typy i parametry symulowanych serwisów wykorzystywanych w ramach scenariusza $\mathcal{B}$	46
6.1	Wartości przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	57
6.2	Wartości przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	58
6.3	Wartości przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	59
6.4	Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	61
6.5	Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	63
6.6	Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	65
6.7	Wartości średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od wariantu serwisów i modelu współbieżności	69

Spis kodów źródłowych

1	Zapytanie Q1 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$	39
2	Zapytanie Q2 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$	40
3	Zapytanie Q3 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$	40
4	Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+PT	41
5	Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie R2DBC	42
6	Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+VT	42
7	Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC w ramach scenariusza $\mathcal{A}$	43
8	Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika JDBC w ramach scenariusza $\mathcal{A}$	44
9	Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Legacy	47
10	Metoda zawierająca właściwą logikę agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Legacy	48
11	Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Reactive	49
12	Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom	50
13	Implementacja własnej strategii agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom	50